



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 97. Modelos de integración continua. Herramientas y sus
aplicaciones**

AUTOR: José Antonio Eusamio Mazagatos.

Actualización 2019

ÍNDICE

1	MODELOS DE INTEGRACIÓN CONTINUA.....	4
1.1.	CONTINUOUS DELIVERY – ENTREGA CONTINUA.....	6
1.1	PRINCIPIOS DE LA ENTREGA CONTINUA	8
1.1.1	INCORPORAR LA CALIDAD INTRÍNSECAMENTE EN EL PROCESO DE CONSTRUCCIÓN..	8
1.1.2	TRABAJAR EN PEQUEÑOS LOTES	8
1.1.3	LOS ORDENADORES REALIZAN TAREAS REPETITIVAS, LAS PERSONAS RESUELVEN PROBLEMAS.....	9
1.1.4	CONTINUAMENTE PERSIGUE LA MEJORA CONTINUA.....	9
1.1.5	TODOS SON RESPONSABLES.....	9
2.	FUNDAMENTOS DE ENTREGA CONTINUA.....	10
2.1.	GESTIÓN DE LA CONFIGURACIÓN	10
2.1.1.	HERRAMIENTAS DE GESTIÓN DE LA CONFIGURACIÓN	12
2.2.	INTEGRACIÓN CONTINUA	12
2.2.1.	PRÁCTICAS DE INTEGRACIÓN CONTINUA.....	13
2.2.2.	MANTENER UN REPOSITORIO DE CÓDIGO ÚNICO.....	14
2.2.3.	AUTOMATIZAR LA CONSTRUCCIÓN	14
2.2.4.	BUILD AUTO-PROBABLE	15
2.2.5.	COMMIT CON LA LÍNEA PRINCIPAL TODOS LOS DÍAS.....	15
2.2.6.	COMMIT DEBE CONSTRUIR LA LÍNEA PRINCIPAL EN UNA MÁQUINA DE INTEGRACIÓN	16
2.2.7.	ARREGLAR LAS CONSTRUCCIONES ROTAS INMEDIATAMENTE	17
2.2.8.	MANTENER EL PROCESO DE CONSTRUCCIÓN RÁPIDO	18
2.2.9.	PROBAR EN UN CLON “PERFECTO” DEL ENTORNO DE PRODUCCIÓN.....	19
2.2.10.	HACER FÁCIL PARA CUALQUIERA OBTENER EL ÚLTIMO EJECUTABLE	19
2.2.11.	TODOS PUEDEN VER LO QUE ESTÁ PASANDO.....	20
2.2.12.	DESPLIEGUE AUTOMÁTICO	20
2.3.	TESTING CONTINUO	21
3.	IMPLEMENTANDO LA ENTREGA CONTINUA.....	26
3.1.	ARQUITECTURA.....	26
3.1.1.	ARQUITECTURA EVOLUTIVA.....	27
3.2.	CULTURA.....	28
3.3.	PATRONES.....	30
3.3.1.	EL CANAL DE DESPLIEGUE (DEPLOYMENT PIPELINE).....	30
3.3.2.	PATRONES PARA LANZAMIENTOS DE BAJO RIESGO	32
3.4.	ENTREGA CONTINUA Y DEVOPS.....	35
3.5.	HERRAMIENTAS DE INTEGRACIÓN CONTINUA.....	36
3.5.1.	HERRAMIENTAS DE CONTROL DE VERSIONES	37
3.5.1.1.	GITHUB.....	37
3.5.1.2.	BITBUCKET	37

3.5.1.3. SUBVERSION (SVN)	37
3.5.2. HERRAMIENTAS DE INTEGRACIÓN CONTINUA.....	38
3.5.2.1. JENKINS	39
3.5.3. REPOSITORIOS DE ARTEFACTOS	40
3.5.3.1. ARTIFACTORY – JFROG	40
3.5.3.2. APACHE ARCHIVA	40
3.5.4. EMPAQUETADO Y DESPLIEGUE DE APLICACIONES	40
3.5.4.1. APACHE MAVEN.....	40
3.5.4.2. GRADLE	40
3.5.5. HERRAMIENTAS DE TESTING	41
3.5.5.1. JMETER	41
3.5.5.2. JUNIT – [X]UNIT	41
3.5.5.3. SOAPUI.....	42
3.5.5.4. SELENIUM.....	42
3.5.6. HERRAMIENTAS DE CONTROL DE CALIDAD ESTÁTICO.....	43
3.5.6.1. SONARQUBE	43
3.5.7. HERRAMIENTAS DE GESTIÓN DE REQUISITOS Y CASOS DE PRUEBAS	44
3.5.7.1. TESTLINK.....	44
4. RESUMEN ESQUEMÁTICO.....	45
5. GLOSARIO.....	48
6. TEST.....	49
6.1 PREGUNTAS DE TEST.....	49
6.2 SOLUCIONES A LAS PREGUNTAS DE TEST	51
7 BIBLIOGRAFÍA	52
7.1 BIBLIOGRAFÍA BÁSICA	52
7.2 BIBLIOGRAFÍA PARA AMPLIAR EL TEMA.....	52

1 Modelos de Integración Continua

La integración continua (continuous integration en inglés) es un modelo informático propuesto inicialmente por Martin Fowler que consiste en hacer integraciones automáticas de un proyecto lo más frecuente posible para poder detectar fallos lo antes posible. Dentro del proceso de desarrollo software se entiende por integración la compilación y ejecución de pruebas de todo un proyecto, donde se juntan trabajos de distintos miembros del equipo. Integrar es “juntar” de manera armoniosa elementos de naturaleza o constitución diversa.

La definición más aceptada, acuñada por el propio Martin Fowler dice que *"la **integración continua** es una práctica de desarrollo de software donde los miembros de un equipo integran su trabajo con frecuencia, por lo general cada persona se integra al menos diariamente, lo que lleva a múltiples integraciones por día. Cada integración se verifica mediante una compilación automatizada (incluidas las pruebas) para detectar errores de integración lo más rápido posible. Muchos equipos encuentran que este enfoque conduce a problemas de integración significativamente reducidos y permite que un equipo desarrolle un software cohesivo más rápidamente"*.

De manera resumida el proceso suele ser: cada cierto tiempo (horas, días), descargarse las fuentes desde el control de versiones (por ejemplo CVS, Git, Subversion, Mercurial o Microsoft Visual SourceSafe) compilarlo, ejecutar pruebas y generar informes. Para esto suelen utilizarse aplicaciones como Solano CI, Bamboo, Pipeline, Apache Continuum, Hudson, Jenkins, GoCD, CruiseControl o Anthill (para proyectos Java) o CruiseControl.Net, Team Foundation Build para .Net, que se encargan de controlar las ejecuciones, apoyadas en otras herramientas como Ant o Maven (para proyectos Java, PHP, etc..), o Nant o MSBUILD (para .Net) que se encargan de realizar las compilaciones, ejecutar las pruebas y realizar los informes. Este flujo se recoge en la siguiente figura de manera genérica, donde más etapas o detalles podrían incluirse.

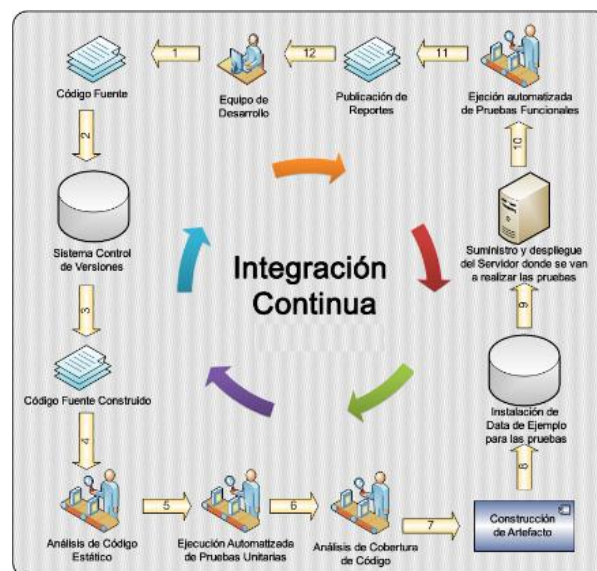


Figura 1Ciclo ilustrativo del flujo de integración Continua

El primer trabajo conocido sobre integración continua fue el entorno Infuse desarrollado por G.E. Kaiser, D.E. Perry y W.M. Schell. [6]

En 1994, Grady Booch utilizó la frase integración continua en el análisis orientado a objetos y el diseño con aplicaciones [7] para explicar cómo, al desarrollar utilizando microprocesos, "las versiones

internas representan una especie de integración continua del sistema, y existen para forzar el cierre del microprocesamiento".

En 1997, Kent Beck y Ron Jeffries inventaron la Programación Extrema (XP) en el proyecto del Sistema de Compensación Integral de Chrysler, incluida la integración continua. [8] Beck publicó sobre la integración continua en 1998, enfatizando la importancia de la comunicación cara a cara sobre el apoyo tecnológico. [9] En 1999, Beck elaboró más en su primer libro completo sobre Programación Extrema. [10] CruiseControl, una de las primeras herramientas de CI de código abierto, [11] se lanzó en 2001.

Por este origen, la integración continua está asociada con las metodologías de programación extrema y desarrollo ágil, aunque no es exclusivo ni excluyente.

Una historia común de los proyectos de software suele ser que la integración es un proceso largo e impredecible. Sin embargo en palabras del propio Fowler este no tiene por qué ser el camino. La integración no tiene por qué ser un "evento" o un miniproyecto, sino que se puede gestionar como tareas "rutinarias". De esta manera el trabajo de cualquier desarrollador individual está a solo unas horas de formar parte de un proyecto compartido y se puede integrar en minutos. Cualquier error de integración se encuentra rápidamente y puede solucionarse rápidamente.

Junto al concepto de integración continua, más recientemente, en torno a 2010 se empieza hablar de manera intensa de "Continuous Delivery" (Entrega Continua) y posteriormente de "Continuous Deployment" (Despliegue Continuo) para acabar englobados todos dentro de uno más amplio que es DevOps o Dev(Sec)Ops.

Estos conceptos no son el resultado de una herramienta costosa y compleja (aunque las herramientas ayudan y para eso están). La esencia del cambio radica en una sencilla práctica, que todos los integrantes del equipo se integren con frecuencia, generalmente a diario, en un repositorio de código fuente controlado.

Varias reacciones se suelen dar ante este planteamiento:

- ✚ Aquí eso no puede funcionar
- ✚ Hacerlo no generará mucha diferencia
- ✚ Ahora no tenemos tiempo, nos parece bien, pero cuando podamos.

Lo que son claros ejemplos de la reacción ante un cambio de paradigma o de mentalidad. Sin embargo, a medida que se intenta aplicar este concepto se puede observar que es mucho más fácil de lo que parece y que supone una gran diferencia en el desarrollo. Es un hecho significativo y relevante que genera en aquellas organizaciones que lo han incorporado otra reacción bien distinta: "sí, lo hacemos, ¿cómo podrías vivir sin eso?"

El término 'Integración continua' se originó con el proceso de desarrollo de programación extrema de Kent Beck, como una de sus doce prácticas originales.

El movimiento Agile se posicionó alejándose de la documentación pesada que imponían los métodos clásicos y orientándose a trabajar con "pedazos de trabajo livianos". Todo el mundo estaba experimentando una revelación colectiva: **el riesgo es inevitable**, por lo que se puede minimizar ese riesgo dividiéndolo en trozos pequeños, es decir, masticar la comida antes de tragarla para evitar ahogarse y morir.

A medida que las personas se sentían más cómodas con el fallo, creaban lugares donde este no solo era aceptable, sino que se fomentaba. El auge de los **frameworks** de prueba como JUnit y Cucumber reflejó esta nueva actitud hacia el riesgo. Las herramientas no preceden a los procesos, las creamos para optimizar los procesos existentes, mejorarlos y evolucionarlos. Esto significaba que los ingenieros pasaban el tiempo suficiente escribiendo pruebas para justificar la creación de estos frameworks.

Extreme Programming fue la principal influencia detrás de este entusiasmo por las pruebas. Kent Beck y Ron Jeffries habían defendido el desarrollo guiado por pruebas (TDD - test-driven development) con un argumento convincente: *"Si los errores van a ocurrir (y lo harán), los ingenieros deberían solucionar al menos algunos de ellos mientras aún recuerdan el código". Debido a que el código se evapora de la mente del ingeniero más rápido que una gota de baijiu.*

Escribir pruebas antes del código obligó a los ingenieros a pensar en cómo querían que funcionara el software. En muchos sentidos, esta práctica sirvió al mismo propósito que la documentación exhaustiva de los años de Waterfall: las pruebas especificaron cómo deberían ser las cosas. Pero las pruebas automatizadas fueron un paso más allá al hacer cumplir estos requisitos en cada cambio de código, mostrando claramente cuando algo no estaba bien.

Los ingenieros descubrieron que, al realizar pruebas con más frecuencia, aumentaban las probabilidades de detectar errores y prevenir catástrofes. Eso significaba comprobar a medida que avanza en lugar de al final. Esto puede parecer obvio hoy, pero en ese momento era una forma fundamentalmente diferente de desarrollar software. Este fue el caldo de cultivo a partir del cual nació la integración continua.

Centraremos el desarrollo del tema en el concepto de "Entrega Continua" al integrar este el de Integración continua como uno de sus pilares básicas. Se analizarán los principios y fundamentos de la entrega continua, para a continuación extender este concepto al de despliegue continuo y hacer un barrido por las distintas herramientas existentes que permiten todos estos pasos.

Se terminará haciendo una introducción al concepto o filosofía DevOps como elemento integrador del modelo de desarrollo ágil con la explotación igualmente ágil y aglutinador de todos los conceptos asociados a "CI/CD/CD".

En el apartado de herramientas de IC se desarrollará un poco más las características de Jenkins y de otros servidores de IC.

1.1. Continuous Delivery – Entrega Continua

La entrega continua es la capacidad de obtener cambios de todo tipo, incluidas nuevas funciones, cambios de configuración, correcciones de errores y experimentos, en producción o en manos de los usuarios, de forma segura y rápida de una manera sostenible.

El objetivo es hacer despliegues y entregas asuntos predecibles y de rutina que se pueden realizar bajo demanda, independientemente de que se trate de un sistema distribuido a gran escala, un entorno de producción complejo, un sistema integrado o una aplicación sencilla.

Logramos todo esto asegurándonos de que el código esté siempre en un estado de "desplegable", incluso frente a equipos de miles de desarrolladores que realizan cambios a diario. De este modo, eliminamos por completo las fases de integración, prueba y aceptación que tradicionalmente seguían "dev complete", así como que el código se congele.

A menudo se asume que si queremos implementar software con más frecuencia, debemos aceptar niveles más bajos de estabilidad y confiabilidad en los sistemas. De hecho, las investigaciones revisadas por pares muestran que este no es el caso: los equipos de alto rendimiento brindan servicios de manera más rápida y confiable que la competencia “de bajo rendimiento”. Esto es cierto incluso en dominios altamente regulados, como los servicios financieros y el gobierno. Esta capacidad proporciona una ventaja competitiva increíble para las organizaciones que están dispuestas a invertir el esfuerzo para lograrlo.

Las prácticas en el corazón de la entrega continua nos ayudan a lograr varios beneficios importantes:

- ✚ **Liberaciones de bajo riesgo.** El objetivo principal de la entrega continua es hacer que los despliegues de software sean eventos indoloros y de bajo riesgo que se puedan realizar en cualquier momento, a pedido. Al aplicar patrones como las implementaciones azul-verdes, es relativamente sencillo lograr implementaciones sin tiempo de inactividad que no son detectables por los usuarios.
- ✚ **Time to Market - Tiempo más rápido al mercado.** No es infrecuente que la integración y la fase de prueba / corrección del ciclo de vida de entrega de software en fases tradicional consuman semanas o incluso meses. Cuando los equipos trabajan juntos para automatizar los procesos de compilación y despliegue, aprovisionamiento de entornos y pruebas de regresión, los desarrolladores pueden incorporar las pruebas de integración y regresión en su trabajo diario y eliminar por completo estas fases. También evitamos las grandes cantidades de trabajo que afectan al enfoque gradual.
- ✚ **Mejor calidad.** Cuando los desarrolladores tienen herramientas automatizadas que descubren regresiones en cuestión de minutos, los equipos se liberan para centrar sus esfuerzos en la investigación de usuarios y en actividades de pruebas de nivel superior, como pruebas exploratorias, pruebas de usabilidad y pruebas de rendimiento y seguridad. Al construir un canal de implementación, estas actividades se pueden realizar de manera continua a lo largo del proceso de entrega, asegurando que la calidad se incorpore a los productos y servicios desde el principio.
- ✚ **Costos más bajos.** Cualquier producto o servicio de software exitoso evolucionará significativamente a lo largo de su vida útil. Al invertir en la compilación, la prueba, la implementación y la automatización del entorno, reducimos sustancialmente el costo de realizar y entregar cambios incrementales al software al eliminar muchos de los costos fijos asociados con el proceso de lanzamiento.
- ✚ **Mejores productos.** La entrega continua hace que sea económico trabajar en pequeños lotes. Esto significa que podemos obtener comentarios de los usuarios a lo largo del ciclo de vida de la entrega en función del software que funcione. Técnicas como las pruebas A / B nos permiten adoptar un enfoque basado en hipótesis para el desarrollo de productos mediante el cual podemos probar ideas con los usuarios antes de desarrollar funciones completas. Esto significa que podemos evitar los 2/3 de las características que creamos que ofrecen un valor cero o negativo a nuestros negocios.
- ✚ **Equipos más felices.** La investigación revisada por pares ha demostrado que la entrega continua hace que las liberaciones sean menos dolorosas y reduce el agotamiento del equipo. Además, cuando lanzamos con mayor frecuencia, los equipos de entrega de software pueden participar más activamente con los usuarios, aprender qué ideas funcionan y cuáles no, y ver de primera mano los resultados del trabajo que han realizado. Al eliminar las actividades dolorosas de poco valor asociadas con la entrega de software, podemos centrarnos en lo que más nos importa: deleitar continuamente a nuestros usuarios.

Si esto suena demasiado bueno para ser verdad, tenga en cuenta que la entrega continua no es magia. Se trata de una mejora diaria y continua: la disciplina constante de lograr un mayor rendimiento siguiendo la heurística "si duele, hágalo más a menudo y haga avanzar el dolor".

1.1 Principios de la Entrega Continua

Hay cinco principios en el corazón de la entrega continua:

- ✚ Incorporar la calidad intrínsecamente en el proceso de construcción (Build quality in)
- ✚ Trabajar en lotes pequeños.
- ✚ Los ordenadores realizan tareas repetitivas, las personas resuelven problemas.
- ✚ Persigue sin descanso la mejora continua.
- ✚ Todos son responsables

Es fácil atascarse en los detalles de la implementación continua (herramientas, arquitectura, prácticas, políticas). Si se encuentra perdido, intente revisar estos principios y puede que le ayude a volver a concentrarse en lo que es importante.

1.1.1 Incorporar la calidad intrínsecamente en el proceso de construcción

W. Edwards Deming, una figura clave en la historia del movimiento Lean, ofreció 14 principios clave para la administración. El principio tres dice: *"Deje de depender de la inspección para lograr la calidad. Elimine la necesidad de una inspección masiva al incorporar la calidad en el producto en primer lugar"*.

Es mucho más barato solucionar problemas y defectos si los encontramos de inmediato, idealmente antes de que se incorporen en el control de versiones, ejecutando pruebas automatizadas localmente. Encontrar defectos a lo largo de la inspección (como las pruebas manuales) requiere mucho tiempo y requiere una clasificación significativa. Luego debemos corregir el defecto, tratando de recordar lo que estábamos pensando cuando presentamos el problema días o incluso semanas atrás.

La creación y evolución de bucles de realimentación para detectar problemas lo antes posible es un trabajo esencial e interminable en la entrega continua. Si encontramos un problema en nuestras pruebas exploratorias, no solo debemos solucionarlo, sino también preguntar: ¿Cómo podríamos haber detectado el problema con una prueba de aceptación automática? Cuando falla una prueba de aceptación, deberíamos preguntar: ¿Podríamos haber escrito una prueba de unidad para detectar este problema?

1.1.2 Trabajar en pequeños lotes

En los enfoques por fases tradicionales de desarrollo software, los pases de desarrollo a pruebas o explotación consisten en lanzamientos completos: meses de trabajo por equipos formados por decenas o cientos de personas, siguiendo un modelo en cascada.

En la entrega continua el enfoque es el opuesto intentando que todos los cambios realizados en el control de versiones lleguen lo más lejos (Cercanos a producción) posible obteniendo una respuesta integral lo más rápido posible.

Trabajar en pequeños lotes tiene muchos beneficios:

- ✚ Reduce el tiempo que se tarda en obtener retroalimentación sobre nuestro trabajo
- ✚ Facilita la clasificación y la solución de problemas
- ✚ Aumenta la eficiencia y la motivación
- ✚ Evita sucumbir a la falacia de los costos irrecuperables.

La razón por la que se trabaja en grandes lotes se debe al gran coste fijo de la gestión de cambios. *Un objetivo clave de la entrega continua es cambiar la economía del proceso de entrega del software para que sea económicamente viable trabajar en lotes pequeños y así poder beneficiarse de sus múltiples ventajas.*

1.1.3 Los ordenadores realizan tareas repetitivas, las personas resuelven problemas

Una de las ideas filosóficas más tempranas de la tradición de Toyota es *jidoka*, que a veces se traduce como “automatización con un toque humano”. El objetivo es que las computadoras realicen tareas simples y repetitivas, como las pruebas de regresión, para que los humanos puedan concentrarse en la resolución de problemas. De esta manera los ordenadores y las personas se complementan.

A muchas personas les preocupa que la automatización los deje sin trabajo. Este no es el objetivo. Nunca habrá una escasez de trabajo en una empresa exitosa. Por el contrario, las personas se liberan del trabajo sin sentido para concentrarse en actividades de mayor valor. Esto también tiene el beneficio de mejorar la calidad, ya que los humanos son más propensos a los errores cuando realizan tareas sin sentido o repetitivas.

1.1.4 Continuamente persigue la mejora continua

La mejora continua, o *kaizen* en japonés, es otra idea clave del movimiento Lean. Taiichi Ohno, una figura clave en la historia de la compañía Toyota, dijo:

*Las oportunidades de Kaizen son infinitas. No creas que has hecho las cosas mejor que antes y estar a gusto ... Esto sería como el estudiante que se enorgullece porque superó a su maestro dos veces de cada tres en esgrima. Una vez que recoja los brotes de las ideas de Kaizen, es importante tener la actitud en nuestro trabajo diario de que **debajo de una idea de Kaizen se encuentra otra.***

No debe tratarse la transformación como un proyecto en el que se puede embarcar y completar para luego volver al negocio habitual. Las mejores organizaciones son aquellas en las que todos consideran el trabajo de mejora como una parte esencial de su trabajo diario y donde nadie está satisfecho con el statu quo.

1.1.5 Todos son responsables

En las organizaciones de alto rendimiento, nada es “problema de otro”. Los desarrolladores son responsables de la calidad y la estabilidad del software que crean. Los equipos de operaciones son responsables de ayudar a los desarrolladores a desarrollar la calidad. Todos trabajan juntos para lograr los objetivos a nivel organizativo, en lugar de optimizar lo mejor para su equipo o departamento.

Cuando las personas realizan optimizaciones locales que reducen el rendimiento general de la organización, a menudo se debe a problemas sistémicos:

- ✚ Sistemas de gestión deficientes
- ✚ Ciclos de presupuesto anual
- ✚ Incentivos que recompensan los comportamientos incorrectos o no óptimos. Un ejemplo clásico es recompensar a los desarrolladores por aumentar su velocidad o escribir más código, y recompensar a los evaluadores en función de la cantidad de errores que encuentren.

La mayoría de las personas quieren hacer lo correcto, pero adaptarán su comportamiento en función de cómo sean recompensados. Por lo tanto, es muy importante crear ciclos rápidos de

retroalimentación de las cosas que realmente importan: ***cómo reaccionan los clientes a lo que construimos para ellos y el impacto en nuestra organización.***

2. Fundamentos de Entrega Continua

La entrega continua se basa en tres pilares o fundamentos:

- ✚ Gestión de configuración integral
- ✚ Integración continua
- ✚ Pruebas continuas.

A continuación se describe cada una de estos pilares.

2.1. Gestión de la configuración

La **Gestión de Configuración de Software** (Software Configuration Management, SCM) es una especialización de la gestión de configuración a todas las actividades del desarrollo software.

SCM trata y controla:

- ✚ La elaboración de código fuente por varios desarrolladores simultáneamente.
- ✚ El seguimiento del estado de las fases del desarrollo de software (versiones) y sus cambios (control de versiones).
- ✚ La conducción de la integración de las partes del software en un solo producto de software.

Se llama **control de versiones** a la gestión de los diversos cambios que se realizan sobre los elementos de algún producto o una configuración del mismo. Una versión, revisión o edición de un producto es el estado en el que se encuentra el mismo en un momento dado de su desarrollo o modificación.

El control de versiones se realiza principalmente en la industria informática para controlar las distintas versiones del código fuente dando lugar a los sistemas de control de código fuente o SCM (siglas del inglés Source Code Management). Sin embargo, los mismos conceptos son aplicables al resto de elementos que forman parte del proceso de desarrollo o incluso a otros ámbitos como documentos, imágenes, sitios web, etc.

Aunque un sistema de control de versiones puede realizarse de forma manual, hoy en día en un proyecto serio es inviable, siendo imprescindible disponer de herramientas que faciliten esta gestión dando lugar a los llamados sistemas de control de versiones o VCS (del inglés Version Control System). Estos sistemas facilitan la administración de las distintas versiones de cada producto desarrollado, así como las posibles especializaciones realizadas (por ejemplo, para algún cliente específico).

La automatización desempeña un papel vital para garantizar que podamos lanzar software de forma repetida y confiable. Un objetivo clave es elegir procesos manuales repetitivos y automatizarlos:

- ✚ Compilación
- ✚ Implementación
- ✚ Pruebas de regresión
- ✚ Aprovisionamiento de infraestructura

Para lograrlo se deben controlar todas las versiones necesarias para realizar estos procesos:

- ✚ Código fuente
- ✚ Scripts de configuración, pruebas e implementación

- ✚ Información de configuración de la infraestructura y de la aplicación
- ✚ Librerías, bibliotecas y paquetes de los que dependa el código de nuestro sistema.

También debe ser sencillo consultar el estado actual e histórico de nuestros entornos.

Se tienen dos objetivos primordiales:

- ✚ **Reproducibilidad:** Se debe poder aprovisionar cualquier entorno de forma totalmente automatizada y saber que cualquier entorno nuevo que se reproduzca desde la misma configuración es idéntico.
- ✚ **Rastreabilidad:** Se debe poder elegir cualquier entorno y determinar con rapidez y precisión las versiones de cada dependencia utilizada para crear ese entorno. También será necesario poder comparar versiones anteriores de un entorno y ver qué ha cambiado entre ellos.

Estas capacidades nos dan varios beneficios muy importantes:

- ✚ **Recuperación de desastres:** cuando algo falla en uno de nuestros entornos, por ejemplo, un fallo de hardware o una brecha de seguridad, se debe poder reproducir ese entorno en un período de tiempo determinista para poder restaurar el servicio.
- ✚ **Auditabilidad:** para demostrar la integridad del proceso de entrega, se debe ser capaz de mostrar el camino hacia atrás desde cada despliegue hasta los elementos de los que proviene, incluida su versión. La gestión integral de la configuración, combinada con las tuberías (Pipeline) de implementación lo permite.
- ✚ **Mayor calidad:** el proceso de entrega del software suele estar sujeto a largos retrasos en espera de que se preparen los entornos de desarrollo, prueba y producción. Cuando esto se puede hacer automáticamente desde el control de versiones, se pueden obtener comentarios (feedback) sobre el impacto de nuestros cambios mucho más rápidamente, lo que permite incorporar calidad al desarrollo software.
- ✚ **Administración de capacidad:** si se quiere agregar más capacidad a los distintos entornos, la habilidad para reproducir o clonar servidores existentes es esencial. Esto permite la escalabilidad horizontal de los sistemas, como por ejemplos los sistemas distribuidos basados en la nube.
- ✚ **Respuesta a los defectos:** cuando se descubre un defecto crítico, o una vulnerabilidad en algún componente de nuestro sistema, se necesita una nueva versión de nuestro software lo más rápido posible. Muchas organizaciones tienen un proceso de emergencia para ir más rápido en estas situaciones omitiendo algunas de las pruebas y tareas de auditorías. Esto presenta un dilema especialmente serio en sistemas críticos para la seguridad. El objetivo debe ser poder utilizar el proceso de liberación normal para soluciones de emergencia, que es precisamente lo que permite la entrega continua, sobre la base de una gestión de configuración integral.

A medida que los entornos se vuelven más complejos y heterogéneos, se vuelve cada vez más difícil lograr estos objetivos. Lograr la reproducibilidad perfecta y la trazabilidad hasta el último byte para un sistema empresarial complejo es imposible (aparte de todo lo demás, todo sistema real tiene un estado). Por lo tanto, una parte clave de la administración de la configuración es trabajar para simplificar nuestra arquitectura, entornos y procesos para reducir la inversión requerida para lograr los beneficios deseados.

Al trabajar para lograr los beneficios siempre se debe comenzar por definir en términos mensurables (medibles) los objetivos a lograr. Esto permite determinar cuál de los posibles caminos para alcanzar el objetivo es probablemente el mejor, y cambiar de dirección o reevaluar nuestras metas si descubrimos que nuestro enfoque sería demasiado costoso o demasiado largo.

2.1.1. Herramientas de gestión de la configuración

La elección de herramientas es un tema complejo, y en muchos casos (a menos que use algo totalmente inadecuado) la elección de herramientas no es el factor crítico para el éxito, aunque todo ayuda.

Es importante analizar las herramientas/tecnologías que se utilizan en la organización y seleccionar aquellas que mejor encajarían desde el punto de vista tanto técnico como organizativo.

Ejemplos de este tipo de herramientas son entre otros: **CVS**, **Subversion**, SourceSafe, ClearCase, Darcs, Bazaar, Plastic SCM, **Git**, SCCS, **Mercurial**, Perforce, Fossil SCM, **Team Foundation Server**, **BitBucket**.

En el apartado relativo a herramientas de Integración continua se referenciarán algunas de las características de varias de ellas.

Independientemente de la herramienta elegida, es importante establecer un objetivo a corto plazo y tratar de lograrlo.

Cada nuevo avance tecnológico, como el uso de contenedores / Cloud / virtualización hace que sea más fácil y económico lograr los beneficios descritos. Es preceptivo recordar que primero son los métodos y después las herramientas, que en sí mismas no son una solución. Es obligatorio aplicar la disciplina de lo comprensible, y mucho sentido común.

2.2. Integración continua

Combinar el trabajo de múltiples desarrolladores es difícil. Los sistemas de software son complejos, y un cambio aparentemente simple y “autocontenido” en un solo archivo puede fácilmente tener consecuencias involuntarias que comprometen la corrección del sistema, por las dependencias entre los distintos elementos del sistema. Como resultado, algunos equipos hacen que los desarrolladores trabajen aislados entre sí en sus propias oficinas, tanto para mantener el trunk maestro como para evitar “que se maten”.

Sin embargo, lo que realmente ocurre con el tiempo es que estas ramas divergen ligera pero constantemente unas de otras. Si bien la combinación de una de estas ramas en la línea principal no suele ser problemática, el trabajo requerido para integrar múltiples ramas de larga duración en la línea principal suele ser doloroso, ya que requiere una cantidad significativa de re-trabajo, ya que se revelan las suposiciones de los desarrolladores que entran en conflicto. Esta situación debe resolverse para poder liberar.

Los equipos que ramas de código independientes durante un largo periodo a menudo requieren congelaciones de código, o incluso fases de integración y estabilización ad-hoc, ya que trabajan para integrar estas ramas antes de un lanzamiento. A pesar de las herramientas modernas, que veremos en los siguientes apartados, este proceso sigue siendo caro e impredecible. En equipos más grandes que unos pocos desarrolladores, la integración de múltiples ramas requiere múltiples rondas de pruebas de integración, regresión y corrección de errores para validar que el sistema funcionará como se esperaba después de estas combinaciones. Este problema se vuelve exponencialmente más grave a medida que aumenta el tamaño de los equipos y las ramas son más longevas.

La práctica de la integración continua se inventó para abordar estos problemas. IC (integración continua) sigue el principio de XP (programación extrema) de que si algo es doloroso, deberíamos hacerlo más a menudo y llevar el dolor hacia adelante. De este modo, los desarrolladores de IC integran todo su trabajo en el trunk (línea principal o maestro) de forma regular (al menos diariamente). Se ejecuta un conjunto de pruebas automatizadas antes y después de la fusión para validar que no se introducen regresiones. Si estas pruebas automáticas fallan, el equipo detiene lo

que está haciendo y alguien soluciona el problema inmediatamente. Este modelo emula el cordel y el Andón del modelo Toyota.

De esta manera se puede asegurar que el software está siempre en un estado de trabajo, “ready” y que las ramas de los desarrolladores no se desvíen significativamente del trunk. Los beneficios de la integración continua son muy importantes: las investigaciones y estudios muestran que conduce a *niveles más altos de rendimiento, sistemas más estables y software de mayor calidad*. Sin embargo, la práctica sigue siendo controvertida, por dos razones principales.

- ✚ **Primero:** requiere que los desarrolladores dividan las funciones grandes y otros cambios en pasos más pequeños e incrementales que puedan integrarse en el trunk. Este es un cambio de paradigma para los desarrolladores que no están acostumbrados a trabajar de esta manera. ***También lleva más tiempo completar funciones grandes.*** Sin embargo, en general, *el objetivo que se persigue no es “optimizar” la velocidad a la que los desarrolladores pueden declarar su trabajo “dev complete” en una rama sino que se promoció en producción funcionando correctamente.* Es decir, revisar, integrar, probar y desplegar los cambios lo más rápido posible. Este proceso es un orden de magnitud más rápido y barato cuando los cambios son pequeños y autónomos, y las ramas en las que viven son efímeras, es decir su tiempo de vida es lo más corta posible. Trabajar en pequeños lotes también garantiza que los desarrolladores reciban comentarios regulares sobre el impacto de su trabajo en el sistema en general, de parte de otros desarrolladores, evaluadores, clientes y pruebas automatizadas de rendimiento y seguridad, lo que a su vez hace que cualquier problema sea más fácil de detectar, clasificar y evaluar y por supuesto arreglar.
- ✚ En segundo lugar, la integración continua requiere un conjunto de ***pruebas unitarias automatizadas y de ejecución rápida.*** Estas pruebas **deben ser lo suficientemente exhaustivas como para dar un buen nivel de confianza** de que el software funcionará como se esperaba, y también se ejecutará en unos minutos o menos. Si las pruebas unitarias automatizadas tardan más en ejecutarse, los desarrolladores no querrán ejecutarlas con frecuencia y serán más difíciles de mantener. Crear conjuntos de pruebas unitarias automatizadas que se puedan mantener es complejo y se realiza mejor a través del desarrollo dirigido por pruebas (TDD), en el que los desarrolladores escriben pruebas automatizadas que fallan antes de implementar el código que hace que se pasen las pruebas. TDD tiene varios beneficios, el más importante es que garantiza que los desarrolladores escriban un código que sea modular y fácil de probar, lo que reduce el costo de mantenimiento de las suites de pruebas automatizadas resultantes. Pero TDD todavía no se practica suficientemente.

A pesar de estas barreras, ayudar a los equipos de desarrollo de software a implementar la integración continua debe ser la prioridad número uno para cualquier organización que desee iniciar el viaje hacia la entrega continua. Al **crear rápidos bucles de retroalimentación** y asegurar que los desarrolladores trabajen en pequeños lotes, CI permite a los equipos desarrollar la calidad en su software, reduciendo así el costo del desarrollo continuo del software y aumentando tanto la productividad de los equipos como la calidad del trabajo que producen.

2.2.1. Prácticas de Integración continua

Conseguir que todo esto funcione sin problemas es, obviamente, algo más que una descripción general del proceso. A continuación se indican prácticas clave que conforman una IC efectiva según Martin Fowler:

- ✚ Mantener un repositorio de código único.
- ✚ Automatizar la construcción
- ✚ Build auto-pronunciado
- ✚ **Commit** con la línea principal todos los días.

- ✚ Cada COMMIT debe construir la línea principal en una máquina de integración
- ✚ Arreglar las construcciones rotas inmediatamente
- ✚ Mantener el proceso de construcción rápido
- ✚ Probar en un Clon del entorno de Producción
- ✚ Hacer fácil para cualquiera obtener el último ejecutable
- ✚ Todos pueden ver lo que está pasando.

2.2.2.Mantener un repositorio de código único.

Se debe poner allí todo aquello que se usa, no sólo el código sino todo lo que necesita para hacer una compilación, incluyendo: scripts de pruebas, archivos de propiedades, esquema de base de datos, scripts de instalación, bibliotecas de terceros y configuraciones de IDE

Una de las características de los sistemas de control de versiones es que le permiten crear múltiples “Branches” para manejar diferentes flujos de desarrollo. Esta es una característica útil, no es esencial, y con frecuencia se usa en exceso y causa problemas al equipo. Mantenga su uso de las ramas al mínimo. En particular, se debe tener una línea principal: una sola rama del proyecto actualmente en desarrollo. Casi todo el mundo debería trabajar fuera de esta línea principal la mayor parte del tiempo. (Las ramas razonables son correcciones de errores de versiones de producción anteriores y experimentos temporales).

En general, se debe almacenar en el control de código fuente todo lo que necesita para construir cualquier cosa, pero nada que realmente construya. Algunas personas mantienen los productos de compilación en control de origen, pero considero que es una indicación de un problema más profundo, generalmente una incapacidad para recrear confiablemente las compilaciones.

2.2.3.Automatizar la construcción

Lograr que las fuentes se conviertan en un sistema en ejecución a menudo puede ser un proceso complicado que involucra la compilación, mover archivos, cargar esquemas en las bases de datos, etc. Sin embargo, como la mayoría de las tareas en esta parte del desarrollo de software, se puede automatizar y, como resultado, se debe automatizar. Pedirle a la gente que escriba comandos extraños o hacer clic en los cuadros de diálogo es una pérdida de tiempo y un caldo de cultivo para los errores.

Los entornos automatizados para las compilaciones son una característica común de los sistemas. El mundo de Unix se ha fabricado durante décadas, la comunidad de Java desarrolló Ant, y luego MAVEN. La comunidad de .NET ha tenido a Nant y ahora tiene MSBuild. Un programador debe asegurarse de que puede construir e iniciar su sistema usando estos scripts usando un solo comando.

Un error común es no incluir todo en la compilación automatizada. La compilación debe incluir sacar el esquema de la base de datos del repositorio y encenderlo en el entorno de ejecución. Esto se recoge en una regla de oro: cualquier persona debería poder traer una máquina virgen, revisar los fuentes del repositorio, ejecutar un único comando y tener un sistema en ejecución en su máquina.

Los scripts de construcción vienen en varios sabores y son a menudo específicos de una plataforma o comunidad, pero no tienen por qué serlo.

Una buena herramienta de compilación analiza lo que se necesita cambiar como parte del proceso, normalmente verificando las fechas de los archivos de origen y objeto y compilar solo si la fecha de origen es posterior.

Una herramienta debe ser capaz de resolver las dependencias por complicadas que sean, dependiendo de lo que necesite, puede necesitar diferentes tipos de cosas para construir. Puede construir un sistema con o sin código de prueba, o con diferentes conjuntos de pruebas. Algunos componentes se pueden construir de forma independiente. Un script de compilación debería permitirle construir objetivos alternativos para diferentes casos.

Está bien que los usuarios de IDEs configuren sus propios archivos de proyecto y los utilicen para el desarrollo individual. Sin embargo, es esencial tener una compilación maestra que se pueda usar en un servidor y se pueda ejecutar desde otros scripts. Esto es así porque la mayoría de los IDE tienen algún tipo de proceso de gestión de compilación dentro de ellos.

Por ejemplo en un proyecto Java aunque los desarrolladores incorporen su IDE, la compilación maestra utiliza Ant o MAVEN para garantizar que se pueda ejecutar en el servidor de desarrollo.

2.2.4.Build auto-probable

Tradicionalmente, una compilación significa compilar, enlazar (linkar) y todo lo necesario para que un programa se ejecute. Un programa puede ejecutarse, pero eso no significa que haga lo correcto. Los lenguajes modernos “tipados” estáticamente pueden detectar muchos errores, pero mucho más se escapan.

Una buena manera de detectar errores de manera rápida y eficiente es incluir pruebas automatizadas en el proceso de compilación. La prueba no es perfecta, por supuesto, pero puede detectar muchos errores, lo suficiente como para ser útil. En particular, el aumento de la Programación Extrema (XP) y el Desarrollo Dirigido por Pruebas (TDD) han hecho mucho para popularizar el código de autoevaluación y, como resultado, muchas personas han visto el valor de la técnica. Aunque recomendables, ninguno es estrictamente obligatorio.

Las pruebas diseñadas con anterioridad al código cubren tanto el papel de explorar el diseño del sistema como la detección de errores.

Para tener código autoevaluable, se necesita un conjunto de pruebas automatizadas que puedan verificar una gran parte de la base de código para detectar errores. Las pruebas deben poder iniciarse con un simple comando y ser autocontroladas. El resultado de ejecutar el conjunto de pruebas debe indicar si alguna prueba falló. Para que una compilación sea autoprovable, el fallo de una prueba debería hacer que la compilación falle.

En los últimos años, el aumento de TDD ha popularizado la familia XUnit de herramientas de código abierto que son ideales para este tipo de pruebas. Las herramientas XUnit son sin duda el punto de partida para que su código sea autoevaluable. También se deben complementar con otras herramientas que se centren en pruebas extremo a extremo, como FIT, Selenium, Sahi, Watir, FITnesse y muchas más.

El objetivo no es contar con pruebas para encontrar todo, las pruebas no prueban la ausencia de errores. Sin embargo, la perfección no es el único punto en el que se obtiene un reembolso por una compilación de autocomprobación. Es preferible pruebas imperfectas ejecutadas con frecuencia que pruebas perfectas que jamás se escriben.

2.2.5.Commit con la línea principal todos los días.

La integración tiene mucho que ver con la comunicación. La integración permite a los desarrolladores informar a otros desarrolladores sobre los cambios que han realizado. La comunicación frecuente permite a las personas saber rápidamente a medida que se desarrollan los cambios.

El único requisito previo para que un desarrollador “haga un commit” con la línea principal es que puedan construir su código correctamente. Esto, por supuesto, incluye pasar las pruebas pertinentes relativas a lo entregado. Al igual que con cualquier ciclo de “Commit”, el desarrollador primero actualiza su copia de trabajo para que coincida con la línea principal, resuelve cualquier conflicto con la línea principal y luego se construye en su máquina local. Si la construcción es correcta, entonces están libres de comprometerse con la línea principal.

Al hacer esto con frecuencia, los desarrolladores descubren rápidamente si hay un conflicto entre dos desarrolladores. La clave para solucionar problemas rápidamente es encontrarlos rápidamente. Con los desarrolladores realizando el commit cada pocas horas, se puede detectar un conflicto al poco de haberse producido, el cambio es pequeño y es fácil de resolver. Los conflictos que no se detectan durante semanas pueden ser muy difíciles de resolver.

El hecho de que construya cuando actualiza su copia de trabajo significa que detecta conflictos de compilación así como conflictos de texto. Como la compilación es auto-probable, también detecta conflictos en la ejecución del código. Estos últimos errores son particularmente incómodos para encontrar si permanecen mucho tiempo sin ser detectados en el código. Como solo hay unas pocas horas de cambios entre commits, solo hay muchos lugares donde el problema podría estar escondido. Además, dado que no ha cambiado mucho, se puede utilizar la depuración por diferencias para encontrar el error. Si los cambios son muy frecuentes una simple comparación de los fuentes afectados puede mostrar una línea borrada o alterada por error o algún cambio del que no se es consciente.

Es recomendable que todos los desarrolladores realicen un commit todos los días o incluso más frecuentemente. Cuanto más frecuente, menos lugares para buscar errores de conflicto y más rápidamente se solucionarán los conflictos.

Los commits frecuentes alientan a los desarrolladores a dividir su trabajo en pequeñas porciones de unas pocas horas cada uno. Esto ayuda a rastrear el progreso y proporciona una sensación de progreso. A menudo las personas inicialmente sienten que no pueden hacer algo significativo en solo unas pocas horas, pero un proceso supervisado y la práctica ayudan a aprender.

2.2.6.COMMIT debe construir la línea principal en una máquina de integración

Usando “COMMIT” diarios, un equipo obtiene “builds” probadas con frecuencia. Esto debería significar que la línea principal se mantiene en un estado saludable. En la práctica, sin embargo, las cosas todavía salen mal.

Hay dos razones principalmente:

- ✚ La falta de disciplina: la gente no hace una actualización y construye antes del commit.
- ✚ La diferencia entre entornos, entre las máquinas de los desarrolladores.

Como resultado, hay que asegurarse de que se realicen compilaciones regulares en una máquina de integración y solo si esta compilación de integración tiene éxito, se debe considerar que el commit se ha realizado. Dado que el desarrollador que realiza el commit es responsable de esto, ese desarrollador debe supervisar la compilación de la línea principal para que pueda solucionarlo si se rompe. **Una regla de oro es que no se debe ir nadie a**

casa hasta que la compilación de la línea principal haya pasado con cualquier confirmación que se haya incorporado, evitará malos rollos y problemas al empezar la jornada siguiente. Hay dos formas principales para garantizarlo:

- ✚ Mediante una compilación manual
- ✚ Mediante un servidor de integración continua.

El enfoque de construcción manual es el más simple de describir. Esencialmente, es una cosa similar a la compilación local que hace un desarrollador antes del commit en el repositorio. El desarrollador se dirige a la máquina de integración, revisa la línea principal (que ahora alberga su último commit) y comienza la compilación de integración. Mantiene un ojo en su progreso, y si la construcción tiene éxito, ha terminado con su responsabilidad.

Un servidor de integración continua actúa como un monitor para el repositorio. Cada vez que un commit contra el repositorio finaliza, el servidor verifica automáticamente las fuentes en la máquina de integración, inicia una compilación y notifica al agente el resultado de la compilación. El interlocutor no termina hasta que recibe la notificación, generalmente un correo electrónico.

No todos prefieren usar un servidor CI. Jim Shore dio una descripción bien argumentada de por qué prefiere el enfoque manual. Estoy de acuerdo con él en que **CI es mucho más que simplemente instalar algún software. Todas las prácticas aquí deben estar en juego para hacer la integración continua de manera efectiva**. Muchos equipos que hacen IC bien encuentran que un servidor de CI es una herramienta útil.

Hay que distinguir también entre hacer construcciones regulares, por ejemplo todas las noches y una compilación continua. La construcción regular no es suficiente para una integración continua. El objetivo de la integración continua es encontrar problemas tan pronto como sea posible. Las construcciones nocturnas significan que los bugs permanecen sin ser detectados durante todo un día antes de que alguien los descubra y esto quizás es demasiado tiempo y lleve mucho tiempo encontrarlos y eliminarlos.

2.2.7. Arreglar las construcciones rotas inmediatamente

Una parte clave de hacer una compilación continua es que si la compilación de la línea principal falla, se debe corregir de inmediato. El punto central de trabajar con CI es que siempre se está desarrollando sobre una base estable conocida. No es algo malo que se rompa la compilación de la línea principal, aunque si ocurre todo el tiempo, sugiere que las personas no tienen el cuidado de actualizar y construir localmente antes de un commit. Sin embargo, cuando se rompe la construcción de la línea principal, es importante que se arregle rápidamente.

Kent Beck decía que **"nadie tiene una tarea de mayor prioridad que arreglar la compilación"**. Esto no significa que todos en el equipo tengan que detener lo que están haciendo para arreglar la construcción, por lo general, solo se necesita un par de personas para que las cosas vuelvan a funcionar. **Significa una priorización consciente de una solución de compilación como una tarea urgente y de alta prioridad**.

A menudo, la forma más rápida de arreglar la compilación es revertir la última confirmación de la línea principal, llevando al sistema a la última compilación conocida. Ciertamente, el equipo no debe tratar de hacer ninguna depuración en una línea principal rota, a menos que la causa de la rotura sea inmediatamente obvia. El proceso sería revertir la línea principal y solucionar el problema en una estación de trabajo de desarrollo.

Cuando los equipos están introduciendo IC, a menudo esta es una de las cosas más difíciles de resolver. Al principio, un equipo puede tener dificultades para adquirir el hábito habitual de

trabajar con compilaciones de líneas principales, especialmente si están trabajando en una base de código existente. Se necesita disciplina y paciencia con el proceso.

2.2.8. Mantener el proceso de construcción rápido

El objetivo principal de la integración continua es proporcionar una respuesta rápida. Nada desespera y quema más que una integración que lleva mucho tiempo. Hay que ser también razonable respecto a que quiere decir mucho tiempo, dado que este es “elástico” y no todos las personas/equipos tienen la misma referencia.

Para la mayoría de los proyectos, sin embargo, la pauta de XP de **una compilación de diez minutos está perfectamente dentro de lo razonable**. La mayoría de nuestros proyectos modernos logran esto.

Vale la pena hacer un esfuerzo concentrado para lograrlo, ya que cada minuto que reduce el tiempo de construcción es un minuto ahorrado para cada desarrollador cada vez que se realiza un commit.

Dado que CI exige confirmaciones frecuentes, esto suma mucho tiempo.

Si estás mirando el tiempo de construcción de una hora, llegar a una versión más rápida puede parecer una perspectiva desalentadora. Incluso puede ser desalentador trabajar en un nuevo proyecto y pensar cómo mantener las cosas rápido.

Para las aplicaciones empresariales, el cuello de botella habitual es la fase de pruebas, especialmente las pruebas que involucran servicios externos, como una base de datos.

Probablemente **el paso más crucial es comenzar a trabajar en la configuración de un canal de implementación (Deployment pipeline)**. La idea detrás de un canal de implementación (también conocida como pipeline build o compilación en etapas (staged build)) es que hay varias compilaciones realizadas en secuencia. El commit de la línea principal desencadena la primera compilación, lo que se define como “commit Build”, que es la compilación que se necesita cuando alguien realiza el commit con la línea principal.

Es crucial equilibrar las necesidades y capacidad de detección de errores y la velocidad para que una buena “commit Build”, sea lo suficientemente estable como para que otras personas puedan trabajar.

Jez Humble y Dave Farley ampliaron estas ideas al tema de la entrega continua, que se puede consultar en el libro Continuous Delivery [1], ganador del premio a la excelencia Jolt en 2011.

Una vez que la compilación es buena, otras personas pueden trabajar en el código con confianza. Sin embargo, existen otras pruebas más lentas que se pueden comenzar a realizar en servidores adicionales.

Un ejemplo simple podría ser un proceso de implementación en dos etapas.

- ✚ La primera etapa haría la compilación y ejecutaría pruebas unitarias “sin incluir base de datos o algún elemento externo” de manera que se ejecuten rápidamente, en tiempo inferior a diez minutos.
- ✚ La segunda etapa ejecutaría un conjunto más extenso de pruebas incluyendo la base de datos real y funcionalidades extremo a extremo, pudiendo tardar varias horas en ejecutarse.

La primera etapa se usaría como el ciclo de CI principal por el equipo, y la segunda etapa se ejecuta cuando se puede, dándole preferencia a una ejecución regular y acordada, por ejemplo todos los días, los fines de semana, etc..

Si esta compilación secundaria falla, podría no implicar parar todo el proceso de desarrollo recordando que la prioridad siempre es arreglar lo que esté mal.

Sobre los fallos de esta segunda compilación, lo ideal sería implementar los test necesarios en la primera etapa para optimizar el tiempo de detección y minimizar el impacto en el equipo.

Es posible que haya casos en los que no hay forma de crear una prueba de ejecución rápida que exponga el error, teniendo que probar esa condición “solamente” en la segunda compilación. Esto debería ser excepcional.

Este principio básico puede extenderse a cualquier número de etapas posteriores que se podrían realizar en paralelo si se dispone de recursos y se quiere o necesita reducir el tiempo de “prueba” sin bajar el nivel de exigencia, introduciendo todo tipo de pruebas automatizadas adicionales, incluidas las pruebas de rendimiento, seguridad, etc., en el proceso de construcción regular.

2.2.9. Probar en un Clon “perfecto” del entorno de Producción

Como se ha comentado anteriormente el objetivo de probar es eliminar, bajo condiciones controladas, cualquier problema que el sistema podría encontrarse en producción. Una parte importante de esto es el entorno en el que se ejecutará el sistema de producción. Si realiza la prueba en un entorno diferente, cada diferencia conlleva el riesgo de que lo que ocurra en el momento de las pruebas no ocurra en producción.

Como resultado, es más que deseable configurar el entorno de pruebas para que sea mimético, lo más preciso posible del entorno de producción. Esto afecta a todo, mismo software de base de datos, con las mismas versiones, misma versión del sistema operativo, todas las bibliotecas que se encuentran en el entorno de producción aunque “no se usen realmente”. Idealmente, incluso las mismas direcciones IP y puertos, y por supuesto el mismo hardware.

La duplicación de algunos entornos de producción puede ser prohibitiva (aunque a menudo aparecen falsos problemas económicos para no duplicar entornos de costos moderados) A pesar de estos límites, el objetivo debería ser duplicar el entorno de producción tanto como se pueda y comprender los riesgos que se están asumiendo por cada diferencia entre prueba y producción.

Las máquinas virtualizadas se pueden guardar con todos los elementos necesarios integrados en la virtualización. Entonces es relativamente sencillo instalar las últimas versiones de compilación y ejecución. Además, esto permite ejecutar varias pruebas en una máquina o simular varias máquinas en una red en una sola máquina. A medida que disminuye la penalización de rendimiento de la virtualización esta opción cada vez es más interesante.

2.2.10. Hacer fácil para cualquiera obtener el último ejecutable

Una de las partes más difíciles del desarrollo de software es asegurarse de construir el software correcto. Hemos encontrado que es muy difícil especificar lo que quieres con anticipación y ser correcto; A las personas les resulta mucho más fácil ver algo que no está del todo bien y decir cómo se debe cambiar. Los procesos de desarrollo ágil explícitamente esperan y aprovechan esta parte del comportamiento humano.

Para ayudar a que esto funcione, cualquier persona involucrada con un proyecto de software debería poder obtener el último ejecutable y ejecutarlo: para demostraciones, pruebas exploratorias o simplemente para ver qué ha cambiado esta semana.

Hacer esto es bastante sencillo, tiene que existir un lugar bien conocido donde cualquiera “acreditado” pueda encontrar el último ejecutable. Se podría incluso tener varios. Por ejemplo, las demostraciones necesitan un software cuyas funcionalidades sean familiares a quien realice la “Demo”, por lo que generalmente vale la pena sacrificar lo último por algo que el demostrador sabe cómo operar.

2.2.11. Todos pueden ver lo que está pasando.

La integración continua tiene que ver con la comunicación, por lo que desea asegurarse de que todos puedan ver fácilmente el estado del sistema y los cambios que se le han hecho.

Una de las cosas más importantes para comunicarse es el estado de la construcción de la línea principal. Por ejemplo con CruiseControl, jenkins y otros tienen una consola de administración web incorporado que muestra si hay una compilación en progreso y cuál fue el estado de la última compilación principal.

En Jenkins se puede ver el estado de las últimas ejecuciones, y se puede configurar cuantas ejecuciones anteriores tener disponibles.

A muchos equipos les gusta que esto sea aún más evidente, pudiendo conectar un monitor al sistema de compilación, verde OK, rojo KO. Indicando no solo el estado de la construcción, sino también cuánto tiempo ha estado en ese estado.

Cada equipo toma sus propias decisiones sobre estos sensores de compilación.

Las páginas web de los servidores de CI proporcionan un historial de cambios, lo que permite a los miembros del equipo tener una idea de la actividad reciente en el proyecto. Incluso desde el punto de vista de gestión, para tener una idea de lo que las personas han estado haciendo y de los cambios en el sistema.

Otra ventaja es que permite a aquellos que no están ubicados en el mismo lugar pueden tener una idea del estado del proyecto.

También es útil para que los grupos agreguen información de compilación de múltiples proyectos, proporcionando un estado simple y automatizado de diferentes proyectos.

No es obligatoria una aplicación web. En general cualquier elemento visual de control puede ser de utilidad, por ejemplo un calendario anual donde cada iteración diario se marque con verde o rojo nos permitiría pasar de una tasa de éxito baja (mucho rojo) a todo verde.

2.2.12. Despliegue Automático

Como hemos visto, para realizar la integración continua se necesitan varios entornos, uno para ejecutar pruebas de confirmación y uno o más para ejecutar pruebas secundarias. Lo ideal sería realizar estas operaciones automáticamente. Por lo tanto, es importante contar con scripts que permitan desplegar la aplicación en cualquier entorno fácilmente.

Una consecuencia natural de esto es que también debe tener scripts que le permitan desplegar en producción con una facilidad similar. Normalmente no se despliega en producción todos los días (aunque cada vez es más frecuente tener el objetivo de 10 despliegues al día que preconiza DevOps), pero la implementación hacerlo automáticamente ayuda a acelerar el proceso y reducir los errores. También es una opción barata, ya que solo usa las mismas capacidades que utiliza para desplegar en entornos de prueba.

Si se despliega en producción, debe considerarse la reversión automática. Es imposible evitar que ocurran errores de vez en cuando, y es bueno poder volver rápidamente al último estado "saludable" conocido. Ser capaz de revertir automáticamente también reduce gran parte de la tensión del despliegue, alentando a las personas a desplegar con más frecuencia y, por lo tanto, ofrecer nuevas funciones a los usuarios rápidamente. (La comunidad de Ruby on Rails desarrolló una herramienta llamada Capistrano que es un buen ejemplo de una herramienta que hace este tipo de cosas).

Una funcionalidad interesante puede ser desplegar en un nodo a la vez, reemplazando gradualmente la aplicación en el transcurso de unas pocas horas. Esto permitiría desplegar una versión de prueba para un subconjunto de usuarios, analizarlo antes de decidir si se despliega para todos los usuarios o se revierte. De esta manera se pueden probar nuevas funcionalidades e interfaces de usuario antes de comprometerse con una elección final.

El despliegue automatizado, vinculado a una buena disciplina de CI, es esencial para hacer que todo el proceso funcione.

2.3. Testing continuo

Testing Continuo, o "Automatización de pruebas o Pruebas automáticas", se produce cuando el probador escribe scripts o algún tipo de código que es lanzado a través de scripting o una aplicación específica para probar el software desarrollado. Este proceso implica automatización de un proceso manual, es decir la prueba automatizada se utiliza para ejecutar recurrentemente, rápida y repetida los escenarios de prueba que se realizaron de forma manual.

Las pruebas de automatización se utilizan para pruebas de regresión así como para probar la aplicación desde el punto de vista de carga, rendimiento y estrés. Aumenta la cobertura de la prueba; mejora la precisión, ahorra tiempo y dinero en comparación con las pruebas manuales.

Una pregunta importante es qué automatizar. Idealmente se desearía todo, pero no es posible automatizar todo en el Software, por distintos motivos, coste, esfuerzo, imposibilidad real, etc..

Si tomamos como referencia una aplicación web, las áreas en las que el usuario puede realizar transacciones, como el formulario de inicio de sesión o los formularios de registro, etc., o cualquier área donde muchos usuarios puedan acceder al Software de forma concurrente son claros candidatos a ser automatizadas.

Además, todos los elementos de la Interfaz de usuario, las conexiones con las bases de datos, las validaciones de campos, etc. se pueden probar de manera eficiente mediante la automatización del proceso manual.

En general la automatización en mayor o menor grado, tal y como se ha expuesto en debe incorporar en todos los proyectos software, si bien se debe valorar a la hora de priorizar teniendo en cuenta los siguientes casos:

-  Proyectos grandes y críticos.
-  Proyectos que requieren probar las mismas áreas frecuentemente.
-  Los requisitos no cambian con frecuencia.
-  Acceso a la aplicación para carga y rendimiento con muchos usuarios virtuales.
-  Software estable con respecto a las pruebas manuales.
-  Disponibilidad de tiempo.

Y como se decía Kent Beck comenzar por lo que más nos duele.

La clave para construir software de calidad es asegurarse de que se puede obtener una respuesta rápida sobre el impacto de los cambios. Tradicionalmente, se ha hecho, y en algunos casos se sigue haciendo, un **uso extensivo de la inspección manual** de los cambios de código y las pruebas manuales para demostrar la corrección del sistema. Este tipo de prueba se realizó normalmente en una fase posterior a "dev complete". Sin embargo, esta estrategia tiene varios inconvenientes:

- ✚ **Las pruebas de regresión manuales llevan mucho tiempo y son relativamente caras** de realizar, creando un cuello de botella que nos impide lanzar software con mayor frecuencia y obteniendo retroalimentación de los desarrolladores semanas (y a veces meses) después de que escribieron el código que se está probando.
- ✚ **Las pruebas e inspecciones manuales no son muy confiables**, ya que las personas son notoriamente deficientes para realizar tareas repetitivas, como las pruebas de regresión manualmente, y es extremadamente difícil predecir el impacto de un conjunto de cambios en un sistema de software complejo a través de la inspección.
- ✚ **Cuando los sistemas evolucionan con el tiempo**, como es el caso de los productos y servicios de software modernos, tenemos que hacer un **esfuerzo considerable** para actualizar la documentación de las pruebas para mantenerla actualizada.
- ✚ Para crear calidad en el software, necesitamos adoptar un enfoque diferente. Nuestro objetivo es ejecutar muchos tipos diferentes de pruebas, tanto manuales como automatizadas, continuamente durante todo el proceso de entrega. Los tipos de pruebas que queremos ejecutar están bien organizados en el diagrama del cuadrante creado por Brian Marick, a continuación:

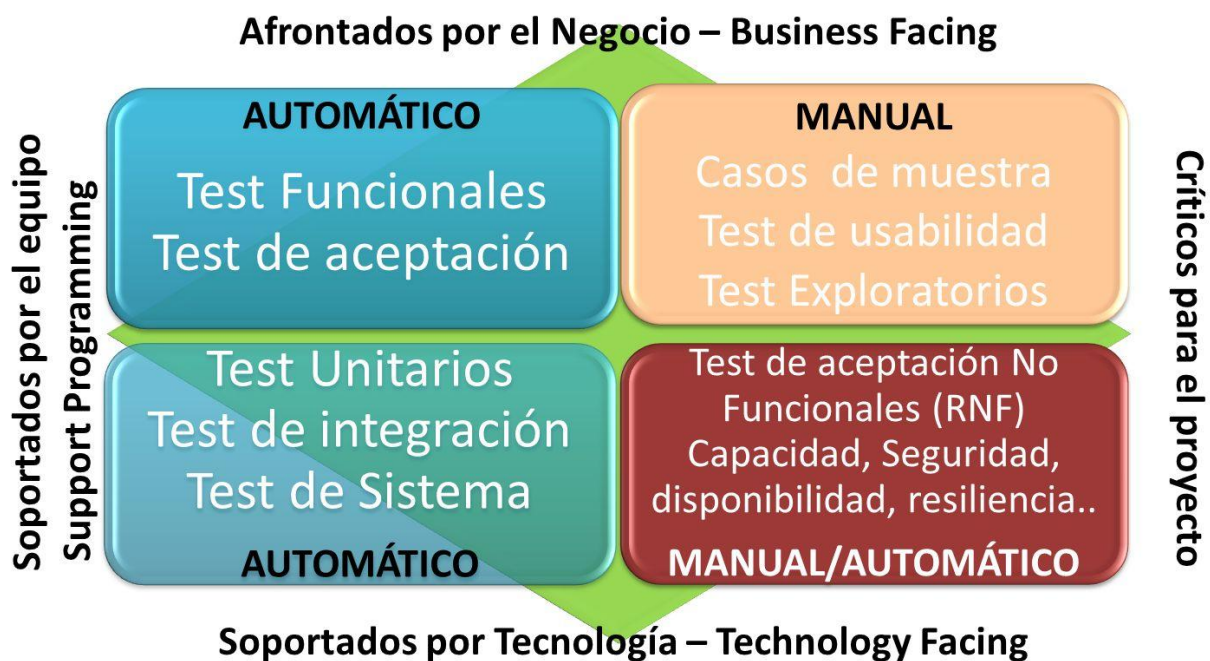


Figura 2.- Diagrama del cuadrante de testing ágil creado por Brian Marick

Una vez que se cuenta con la integración continua y con la automatización de pruebas, se **crea un canal de implementación** (el patrón clave en la entrega continua). En el patrón de canalización de implementación, cada cambio ejecuta una compilación que:

- a) crea paquetes que se pueden implementar en cualquier entorno
- b) ejecuta pruebas unitarias (y posiblemente otras tareas como el análisis estático)

dando retroalimentación a los desarrolladores en el espacio de unos pocos minutos. Los paquetes que pasan este conjunto de pruebas tienen pruebas de aceptación automatizadas más completas que se ejecutan contra ellos. Una vez que tenemos paquetes que pasan todas las pruebas automatizadas, están disponibles para el autoservicio en otros entornos para actividades tales como

pruebas exploratorias, pruebas de usabilidad y, en última instancia, versiones. Los productos y servicios complejos pueden tener tuberías (pipeline) de despliegue sofisticadas; A continuación se muestra una tubería lineal simple:

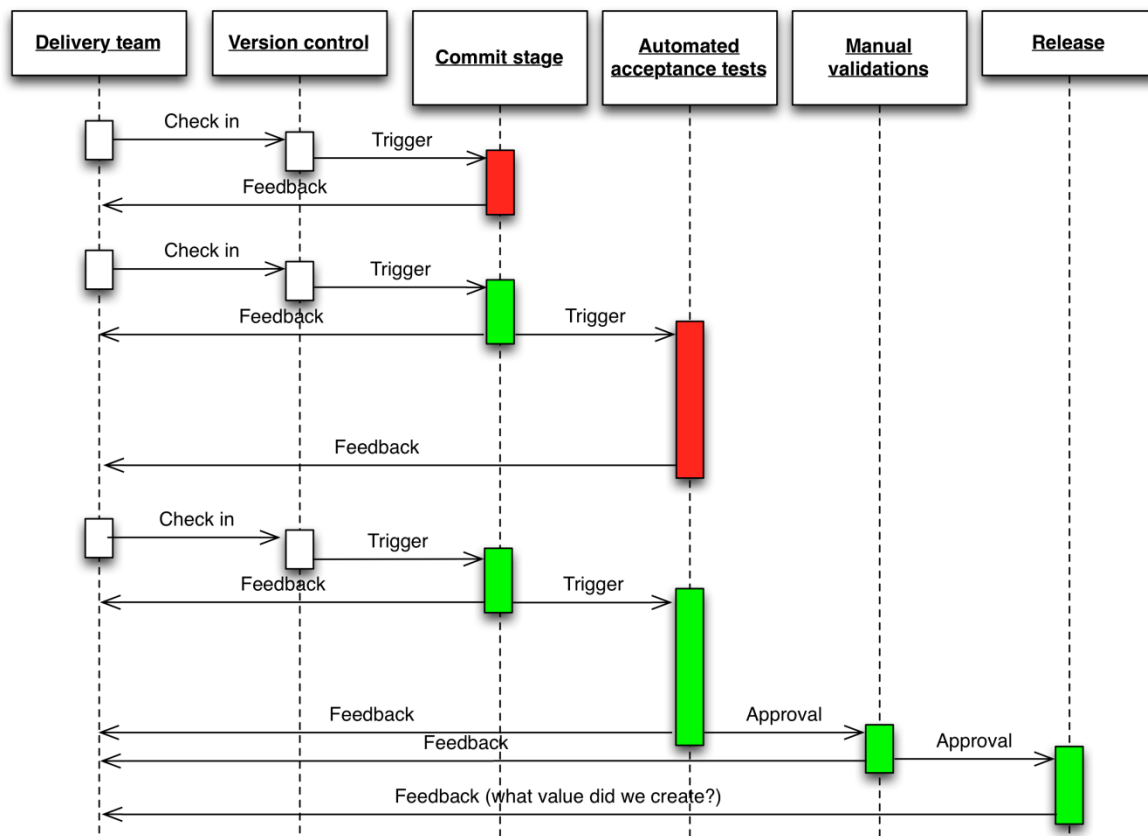


Figura 3 .- Ejemplo de Pipeline de Integración Continua de Jez Humble (Continuous deliver)

En el canal de despliegue, cada cambio es efectivamente un candidato de lanzamiento. El trabajo de la tubería de implementación es detectar problemas conocidos. Si no se detecta ningún problema conocido, se debería liberar cualquier paquete que haya pasado por él con tranquilidad. Si no se hace, o se descubren defectos más adelante, significa que se debe mejorar la tubería, tal vez agregando o actualizando algunas pruebas.

El objetivo debe ser encontrar los problemas lo antes posible y hacer que el tiempo de espera desde el registro en el control de versiones hasta la liberación del código sea lo más breve posible. Por lo tanto, se deben paralelizar las actividades en el canal de implementación, no tener muchas etapas ejecutándose en serie.

También hay un proceso de retroalimentación:

- ✚ Si se descubren errores en las pruebas exploratorias, se debe buscar mejorar las pruebas automatizadas.
- ✚ Si se descubre un defecto en las pruebas de aceptación, se debe buscar mejorar las pruebas unitarias (la mayoría de nuestros defectos deben descubrirse a través de pruebas unitarias).

Se debe comenzar por construir un esqueleto de canal de implementación y unirlos:

- ✚ una prueba unitaria
- ✚ una sola prueba de aceptación,
- ✚ un script de despliegue automatizado
- ✚ un entorno de pruebas exploratorias

Posteriormente se irá aumentando la cobertura de las pruebas y ampliando el canal de implementación a medida que evoluciona el sistema.

Conviene siempre tener en cuenta la pirámide de Mike Cohn que invierte el esfuerzo desde las pruebas manuales a las pruebas automáticas para obtener un mayor ROI (Return On Investment).

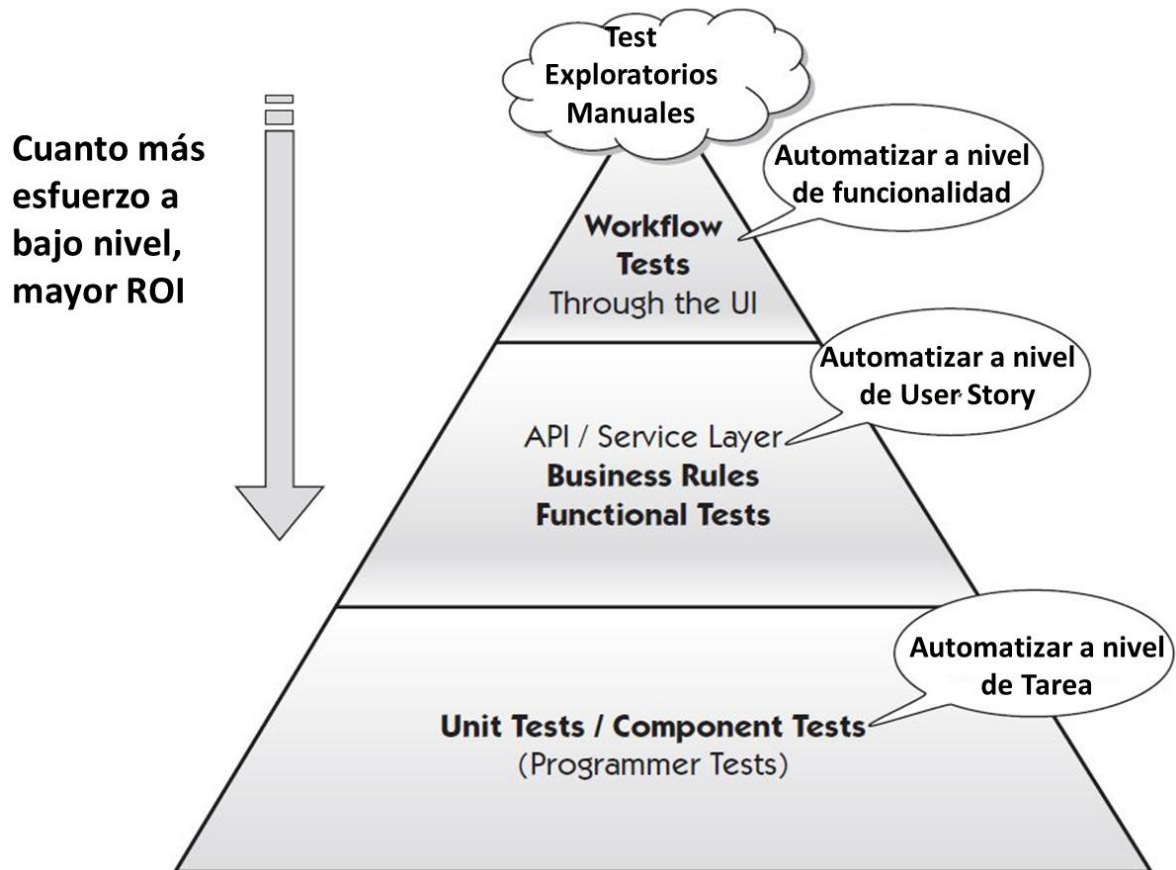


Figura 5. - pirámide de Mike Cohn ROI Vs Esfuerzo por tipo de Test

La automatización se realiza mediante el uso de un lenguaje informático de apoyo o scripting y una aplicación de software automatizada. Hay muchas herramientas disponibles que pueden usarse para escribir scripts de automatización. Antes de mencionar las herramientas, podemos identificar el proceso que se puede utilizar para automatizar las pruebas:

- ✚ Identificación de áreas dentro de un software para automatización.
- ✚ Selección de herramienta adecuada para automatización de pruebas.
- ✚ Escritura de scripts o código de test.
- ✚ Desarrollo de conjuntos de prueba.
- ✚ Ejecución de scripts
- ✚ Crear informes de resultados de ejecución de las pruebas
- ✚ Identificar cualquier error potencial o problema de rendimiento.

En el proceso de evolución hacia el testing continuo, pueden surgir cuestiones organizativas relativas a los equipos actuales y como recorrer ese camino.

La entrega continua no significa disparar a todos nuestros probadores

Los testers tienen una perspectiva única del sistema: entienden cómo los usuarios interactúan con él. **Es recomendable que los testers se emparejen con los desarrolladores (en persona) para ayudarlos a crear y evolucionar los conjuntos de pruebas automatizadas.** De esta manera, los desarrolladores pueden comprender la perspectiva de los testers y estos

aprenden respecto de la automatización de pruebas. Los testers también deben realizar pruebas exploratorias continuamente como parte de su trabajo. Los testers tendrán que aprender nuevas habilidades, como cualquier persona que trabaje en la industria del software.

Que pruebas se deben automatizar y cuales NO

Como se muestra en el diagrama del cuadrante de Brian Marick, ampliamente referenciado por muchos autores como por Lisa Crispin y Janet Gregory (tienen dos grandes libros sobre pruebas ágiles: Pruebas ágiles y Más pruebas ágiles referenciados en la bibliografía) hay actividades manuales importantes, como pruebas exploratorias y pruebas de usabilidad (aunque la automatización puede ayudar con estas actividades, no puede reemplazar a las personas). Se debe apuntar a incorporar todas las actividades de prueba, incluidas las pruebas de seguridad, al proceso de desarrollo y realizarlas continuamente desde el comienzo del ciclo de vida de la entrega del software para cada producto y servicio que se desarrolle.

No se debe detener y automatizar todas las pruebas manuales “de repente”

El proceso de transición debe ser paulatino y constante, basado en la filosofía LEAN de mejora continua (KAIZEN) y de compromiso por todo el equipo. Se recomienda empezar por un par de pruebas automatizadas, las que validan la funcionalidad más importante del sistema para ser ejecutadas en cada COMMIT. Luego se irán agregando nuevas pruebas para cubrir nuevas funcionalidades o cambios en funcionalidades. Con el tiempo, evolucionará un conjunto completo de pruebas automatizadas. En general, es mejor tener 20 pruebas que se ejecuten rápidamente y que el equipo confíe en ellas, que 2.000 pruebas que fallan constantemente y que el equipo ignora.

Todo el equipo es responsable de las pruebas automatizadas

Todo el equipo, en particular los desarrolladores, deben participar en la creación y el mantenimiento de las pruebas automatizadas, y deben detener lo que están haciendo y corregirlas siempre que haya un error. Esto es esencial porque enseña a los desarrolladores cómo escribir software comprobable. Cuando las pruebas automatizadas son creadas y mantenidas por un grupo diferente de los desarrolladores, no hay ninguna fuerza que actúe sobre los desarrolladores para ayudarles a escribir software que sea fácil de probar. La actualización de las pruebas automatizadas en dichos sistemas es dolorosa y costosa, y el software mal diseñado que es difícil de probar es un factor importante que contribuye a las suites de pruebas automatizadas que son costosas de mantener.

El siguiente es un listado (no exhaustivo) de herramientas que se pueden utilizar para las pruebas de automatización:

- ✚ [X]Unit
- ✚ HP Quick Test Professional
- ✚ Selenium
- ✚ IBM Rational Functional Tester
- ✚ SilkTest
- ✚ TestComplete
- ✚ Testing Anywhere
- ✚ WinRunner
- ✚ LoadRunner
- ✚ Visual Studio Test Professional
- ✚ WATIR

3. Implementando la entrega continua

Las organizaciones que intentan implementar la entrega continua tienden a cometer dos errores comunes. El primero es tratar la entrega continua como un estado final, un objetivo en sí mismo. El segundo es gastar mucho tiempo y energía preocupándose sobre qué productos usar.

En esta sección se analizan los dos mayores impedimentos para la entrega continua: la arquitectura empresarial y la cultura organizacional. También se analizarán un conjunto de patrones que se han aplicado con éxito para aumentar el rendimiento, la estabilidad y la calidad.

3.1. Arquitectura

En el contexto de una arquitectura empresarial, generalmente preocupan varios atributos, por ejemplo, la disponibilidad, la seguridad, el rendimiento, la facilidad de uso, etc. En la entrega continua, se introducen dos nuevos atributos arquitectónicos: capacidad de prueba y de despliegue.

- ✚ En una **arquitectura comprobable** (testable), se diseña el software para que la mayoría de los defectos puedan (al menos en principio) ser descubiertos por los desarrolladores al ejecutar pruebas automatizadas en sus equipos. No debe dependerse de entornos complejos e integrados (acoplados) para realizar la mayoría de las pruebas de aceptación y regresión.
- ✚ En una **arquitectura desplegable** (deployable), los despliegues de un producto o servicio en particular se pueden realizar de forma independiente y de forma totalmente automatizada, sin la necesidad de niveles significativos de orquestación. Los sistemas **desplegables** generalmente se pueden actualizar o reconfigurar con un tiempo de inactividad mínimo o nulo.

Cuando no se priorizan la capacidad de prueba y la capacidad de implementación, ocurre que muchas pruebas requieren el uso de entornos complejos e integrados, y las implementaciones son eventos "big bang" que requieren que muchos servicios se liberen al mismo tiempo debido a interdependencias complejas. Estas implementaciones de "big bang" requieren que muchos equipos trabajen juntos de manera cuidadosamente orquestada, con muchas transferencias (Hand-offs) y dependencias entre cientos o miles de tareas. Estas implementaciones suelen tardar muchas horas o incluso días, y requieren programar y gestionar un tiempo de inactividad o pérdida de servicio significativo.

El diseño para la capacidad de prueba y el despliegue comienza con asegurar que los productos y servicios estén compuestos de componentes o módulos bien encapsulados y con poco acoplados (en el contexto de la programación orientada a objetos, tales sistemas siguen los principios del diseño **SOLID**).

Se define una arquitectura modular bien diseñada como una en la que es posible probar o implementar un solo componente o servicio por sí mismo, con cualquier dependencia reemplazada por una **prueba doble** adecuada, que podría tener la forma de una máquina virtual, un stub, o un mock. Cada componente o servicio debe desplegarse de forma totalmente automatizada en los equipos del desarrollador, entornos de prueba o en producción. En una arquitectura bien diseñada, es posible obtener un alto nivel de confianza en que el componente está funcionando correctamente cuando se despliega de esta manera.

Prueba doble es un término genérico para cualquier caso en el que se reemplace un objeto de producción con fines de prueba. Hay varios tipos de dobles pruebas que Gerard Meszaros enumera:

- ✚ Los **objetos ficticios (dummy)** se pasan, pero nunca se usan. Por lo general, sólo se utilizan para rellenar las listas de parámetros.
- ✚ Los **objetos falsos (fake)** en realidad tienen implementaciones de trabajo, pero usualmente toman un atajo que los hace no adecuados para la producción (un ejemplo es *InMemoryTestDatabase*).

- ✚ Los **stubs** proporcionan respuestas enlatadas a las llamadas realizadas durante la prueba, por lo general, no responden en absoluto a nada fuera de lo que está programado para la prueba.
- ✚ Los **spies** son un tipo de *stubs* que también registran cierta información en función de cómo fueron llamados. Una forma de esto podría ser un servicio de correo electrónico que registra cuántos mensajes se enviaron.
- ✚ Las **simulaciones (Mocks)** están preprogramadas con expectativas que forman una especificación de las llamadas que se espera recibir. Pueden lanzar una excepción si reciben una llamada que no esperan y se verifican durante la verificación para asegurarse de que recibieron todas las llamadas que esperaban.

Desde el punto de vista de escalabilidad, la siguiente gráfica descriptiva definida por Martin Fowler muestra la relación entre el crecimiento de nuevas funcionalidades y la calidad del diseño apoyándose en el concepto de entrega continua.

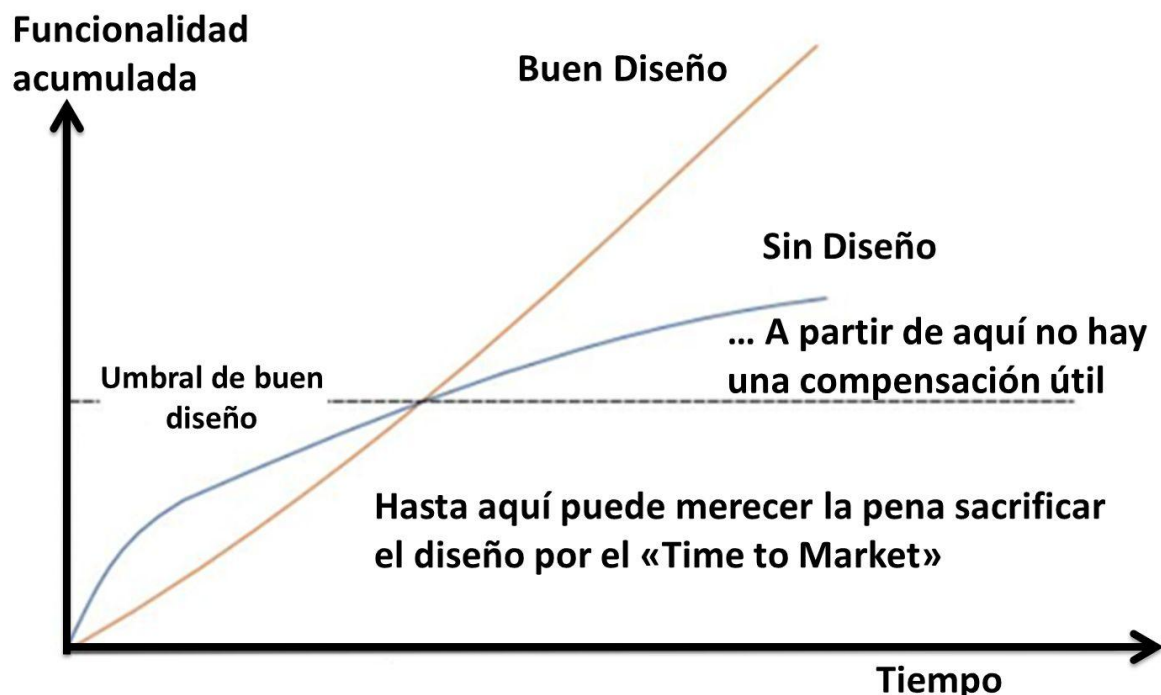


Figura 4 .- Evolución del coste de incorporar nuevas funcionalidades a lo largo del tiempo

Para ayudar al despliegue independiente de componentes, también debe invertirse en la creación de API's versionadas que tengan compatibilidad con versiones anteriores. Asegurar la compatibilidad con versiones anteriores para las API agrega complejidad a nuestros sistemas, pero la flexibilidad que se obtiene en términos de facilidad de implementación compensará esto con creces.

Cualquier arquitectura orientada a servicios debería tener estas propiedades, pero desafortunadamente muchas no las tienen. Sin embargo, el movimiento de microservicios ha priorizado explícitamente estas propiedades arquitectónicas.

3.1.1. Arquitectura evolutiva

Por supuesto, muchas organizaciones viven en un mundo donde los servicios son claramente difíciles de probar y desplegar. En lugar de rediseñar todo, es recomendable un enfoque iterativo para mejorar el diseño del sistema empresarial, a veces conocido como arquitectura evolutiva. En el paradigma de arquitectura evolutiva, se acepta que los productos y servicios exitosos requerirán una nueva arquitectura durante su ciclo de vida debido a los requisitos cambiantes que se les imponen.

Un patrón que es particularmente valioso en este contexto es un estrangulador de aplicación. En este patrón, reemplazamos de forma iterativa una arquitectura monolítica por una más componentizada. Al asegurarnos de que el nuevo trabajo se realice siguiendo los principios de una arquitectura orientada a servicios (SOA), mientras que aceptamos que la nueva arquitectura puede delegar en el sistema al que está reemplazando. Con el tiempo, se ejecutará cada vez más funcionalidad en la nueva arquitectura, y el sistema antiguo que se reemplaza se "estrangula" y desaparece.

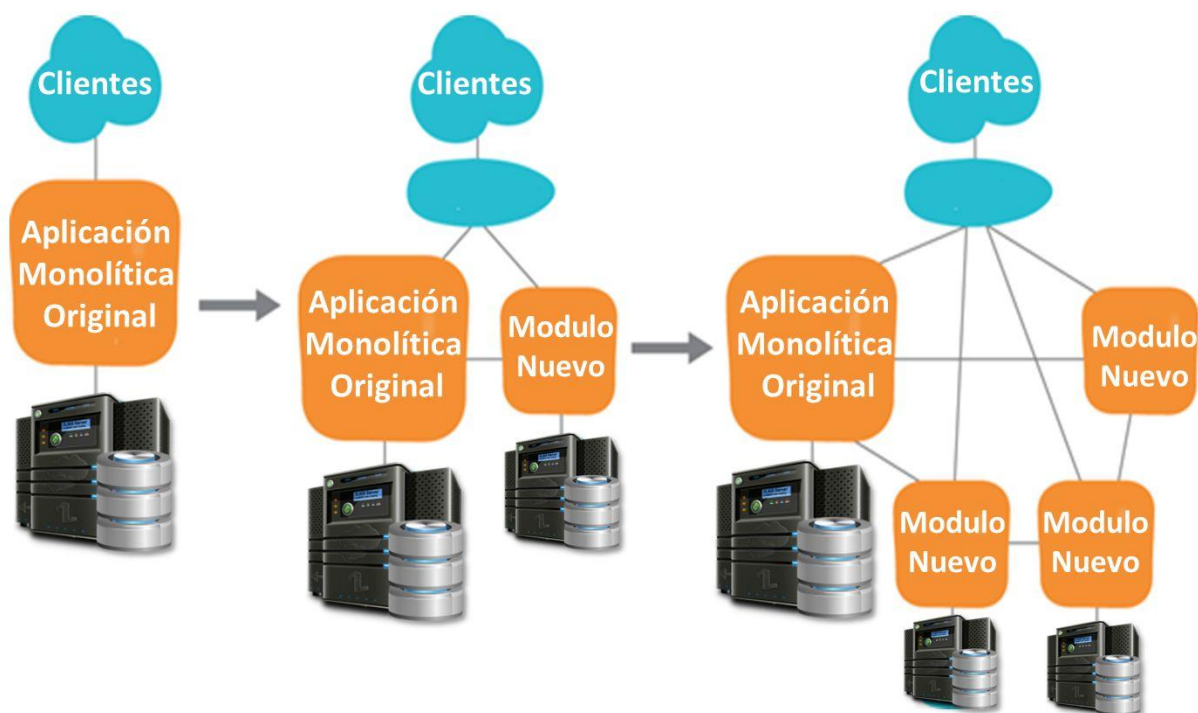


Figura 5.- Ejemplo de arquitectura evolutiva

La Entrega continua se puede aplicar a todos los tipos de arquitectura y desarrollos, desde firmware a mainframes. Podría parecer anecdótico que algunos informes indicaban que era el software empaquetado / COTS en el que la gente de arquitectura parece tener más problemas. Sin embargo, las investigaciones del Informe del Estado de Devops de 2015 y posteriores muestran que no existe una correlación entre el trabajo en este tipo de sistema y el bajo rendimiento.

3.2. Cultura

La cultura corporativa constituye el conjunto de normas, valores y pautas de conducta que comparten los integrantes de la empresa, que se ve reflejado en sus comportamientos. Es determinada por factores tales como: la personalidad de sus directivos y empleados, por la historia de la compañía, el entorno social donde se desarrolla, etc. La cultura es un factor de integración, ya que los miembros del grupo para sentirse parte deben cumplir con las pautas establecidas. Se puede trabajar y gestionar sobre la misma para alinearla con la filosofía y lograr una identidad coherente en todos los sentidos.

La cultura es un conjunto de valores clave, creencias y entendimientos que son compartidos por los miembros de una organización de acuerdo con Smircich, 1983. Pettigrew (1979) describe la cultura organizacional como "el sistema de significados aceptados públicamente y colectivamente operando para un grupo dado en un tiempo dado".

Edgar Schein, autor de "The Corporate Culture Survival Guide", define la cultura como "un patrón de supuestos tácitos compartidos que aprendió un grupo al resolver sus problemas de adaptación externa y de integración interna, que ha funcionado lo suficientemente bien como para ser considerado válido y, por lo tanto, ser enseñados a los nuevos miembros como la forma correcta de percibir, pensar y sentir en relación con esos problemas".

La cultura es intangible y difícil de cambiar, pero puede medirse y es de vital importancia para los resultados de su empresa. La investigación publicada en el Informe del Estado de DevOps de 2017 muestra que, **en el contexto de TI, la satisfacción laboral es el principal factor predictivo de rentabilidad, participación de mercado y productividad**. El mayor predictor de la satisfacción laboral, a su vez, es qué tan efectivamente las organizaciones procesan la información, según lo determinado por un modelo creado por el sociólogo Ron Westrum, que se muestra a continuación:

Patológica (Orientada al Poder)	Burocrática (Orientada a las reglas)	Generadoras (Orientadas al rendimiento)
Baja Cooperación	Cooperación Modesta	Alta cooperación
Se mata al Mensajero	Se culpa al mensajero	Se entrena (fomenta) al mensajero
Se eluden las responsabilidades	Se limitan las responsabilidades	Los riesgos se comparten
Se desincentiva la colaboración puentes entre unidades	Se tolera la colaboración/ puentes entre unidades	Se fomenta la colaboración y los puentes entre unidades
El Fallo conduce a buscar un cabeza de turco	El fallo conduce a la "Justicia ciega"	El fallo conlleva a preguntarse porqué se ha fallado y mejorar
La novedad es machacada y pulverizada	La novedad/Innovación implica problemas	La innovación y la novedad se implementa.

Figura 6.- Tipos de cultura organizativa de Ron Westrum

La investigación de Westrum enfatiza la importancia de crear una cultura donde las nuevas ideas sean bienvenidas, donde personas de toda la organización colaboren en la búsqueda de objetivos comunes, donde se capacite a las personas para que traigan malas noticias y así podamos actuar, y donde se traten los errores y los accidentes como oportunidades para aprender a mejorar en lugar de cazar brujas.

El movimiento DevOps siempre ha enfatizado la importancia primordial de la cultura, con un enfoque particular en la colaboración efectiva entre los equipos de desarrollo y los equipos de operaciones de TI. La investigación muestra que una relación ganar-ganar entre el desarrollo y las operaciones es un factor predictivo significativo del rendimiento de TI. Los profesionales en el movimiento DevOps también han utilizado una serie de herramientas para ayudar a las organizaciones a procesar la información de manera más efectiva, como chatops, autopsias postmortem sin pretender culpabilizar a nadie y la gestión integral de la configuración.

De hecho, las empresas con los mejores rendimientos no esperan que ocurran cosas malas para aprender cómo mejorar, crean accidentes (controlados) de forma regular para aprender más rápido que la competencia. Netflix llevó esto a un nuevo nivel con su "Ejército Simiano", que está rompiendo constantemente su infraestructura para probar continuamente la resistencia de sus sistemas.

El tema común que se puede observar en las organizaciones de alto rendimiento es que siempre están tratando de mejorar. En lugar de esperar a que el entorno cambie a su alrededor o compararse con otros, se consideran a sí mismos como el estándar a vencer.

3.3. Patrones

El concepto de patrones proviene del trabajo del arquitecto Christopher Alexander, quien observa "Cada patrón describe un problema que se presenta una y otra vez en nuestro entorno y luego describe el núcleo de la solución a ese problema de tal manera que puede usar esta solución un millón de veces sin haberlo hecho de la misma manera dos veces".

Linda Rising define un patrón como "una estrategia con nombre para resolver un problema recurrente".

En este apartado vamos a ver dos tipos de patrones o aproximaciones y algunos relacionados con ellos necesarios y recomendables para implementar la entrega continua.

-  Canal de despliegue (Deployment Pipeline)
-  Patrones para lanzamientos de bajo riesgo

3.3.1. El canal de despliegue (Deployment Pipeline)

El patrón clave introducido en la entrega continua es el canal de implementación. Este patrón surgió de varios proyectos de ThoughtWorks a partir de malas experiencias con procesos manuales complejos, frágiles y dolorosos para preparar entornos de prueba y producción y para implementar compilaciones en ellos. El contar con una cantidad significativa de pruebas de regresión y aceptación automatizadas no impedía las demoras de semanas en desarrollar entornos integrados para realizar pruebas de regresión completas.

Para industrializar el proceso de llevar los cambios del control de versiones a producción, el objetivo debe ser que el despliegue en cualquier entorno sea un proceso completamente automatizado y programado, que se pueda realizar bajo demanda en cuestión de minutos. Para ello sería deseable poder configurar entornos de prueba y producción únicamente a partir de archivos de configuración almacenados en el control de versiones. A este concepto se le denomina tuberías de despliegue (deployment pipeline) descrito por Dan North, Chris Read y Jez Humble en un artículo presentado en la conferencia Agile 2006.

En el patrón de canalización de despliegue, cada cambio en el control de versión desencadena un proceso (generalmente en un servidor de IC) que crea paquetes desplegados y ejecuta pruebas unitarias automatizadas y otras validaciones como el análisis de código estático. Este primer paso está optimizado de modo que solo demore unos minutos en ejecutarse. Si esta etapa inicial de confirmación falla, el problema debe solucionarse de inmediato: nadie debe realizar más trabajos en una etapa de confirmación interrumpida. Cada etapa de COMMIT que pasa desencadena el siguiente paso en la tubería, que podría consistir en un conjunto más completo de pruebas automatizadas. Las versiones del software que pasan todas las pruebas automatizadas se pueden desplegar bajo demanda en etapas adicionales, como pruebas exploratorias, pruebas de rendimiento, stage o pre-explotación y producción, como se muestra a continuación.

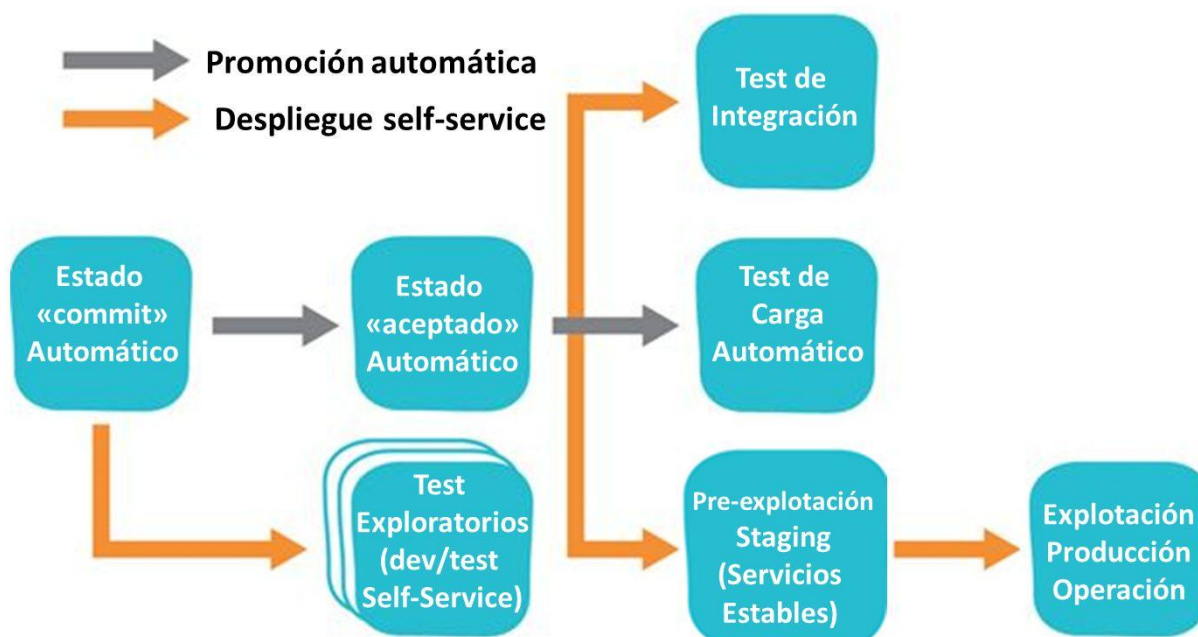


Figura 7 .- Ejemplo de pipeline de Entrega Continua

Las tuberías de implementación unen la administración de la configuración, la integración continua y la automatización de la prueba y la prueba de una manera integral y poderosa que funciona para mejorar la calidad del software, aumentar la estabilidad y reducir el tiempo y el costo requeridos para realizar cambios incrementales al software, sea cual sea el dominio que esté operando.

El canal de implementación mostrado en la imagen anterior puede variar en cuanto a sus opciones y reglas de promoción de un estado a otro dependiendo del estado de madurez de la organización, la criticidad del software y su impacto en el negocio y otros factores internos de la organización.

Es conveniente que las reglas que regulen el proceso de transición de un modelo más tradicional a uno completamente optimizado sean compartidas y asumidas por todos, como se planteaba en los apartados anteriores relativos a la integración continua. Estas reglas o políticas podrían cambiar y evolucionar dependiendo del grado de madurez de la organización.

Cuando se construye un canal de implementación, las siguientes prácticas son valiosas:

- ✚ **Solo construir paquetes una vez.** Es crucial asegurarse de que lo que se está desplegando es lo mismo que se ha probado en todo el proceso de desarrollo, por lo que si falla un despliegue se puede eliminar el Software (packages/componentes) como la fuente del error.
- ✚ **Despliegue de la misma manera en todos los entornos,** incluido el desarrollo. De esta manera, se prueba el proceso de despliegue muchas veces antes de que llegue a producción, y nuevamente, podemos eliminarlo como la fuente de cualquier problema.
- ✚ **Realiza una prueba de humo (smoke test) en todos los despliegues.** Debe haber un script que valide todas las dependencias de su aplicación que están disponibles, en la ubicación donde ha configurado su aplicación. Debe asegurarse de que la aplicación se esté ejecutando y esté disponible como parte del proceso de implementación.
- ✚ **Mantenga sus entornos similares.** Aunque pueden diferir en la configuración del hardware, (Capacidad de proceso) deben tener la misma versión del sistema operativo y los paquetes de middleware, y deben configurarse de la misma manera. Esto se ha vuelto mucho más fácil de lograr con la virtualización moderna y la tecnología de contenedores.

Con el advenimiento de la infraestructura como código, se ha hecho posible utilizar los procesos de despliegue para crear un proceso totalmente automatizado para realizar todo tipo de cambios, incluidos los cambios de la base de datos y la infraestructura, desde el control de versiones hasta la producción de forma controlada, repetible y auditable. Este patrón también se ha aplicado con éxito en el contexto del software instalado por el usuario (incluidas las aplicaciones), el firmware y los mainframes. En el libro “The Practice of Cloud System Administration: DevOps and SRE Practices for Web Services” Thomas A. Limoncelli al sistema resultante se le denomina como una plataforma de entrega de software.

Las tuberías de implementación se describen detalladamente en el Capítulo 5 del libro “Continuous Delivery [1].

3.3.2. Patrones para lanzamientos de bajo riesgo

En el contexto de los sistemas basados en la web, hay una serie de patrones que se pueden aplicar para reducir aún más el riesgo de implementaciones. Michael Nygard también describe una serie de importantes patrones de diseño de software que son fundamentales para crear sistemas resistentes a gran escala.

En contra de lo que intuitivamente se pudiera pensar, el aumento del rendimiento y la mayor estabilidad de los sistemas en producción no son un juego de suma cero. De hecho el despliegue continuo reduce el riesgo de cualquier despliegue o versión individual. Jez Humble (y se confirma en los sucesivos informes del estado del Arte DevOps) indica que "para hacerlo bien" se puede reducir a cuatro principios clave para permitir las versiones de bajo riesgo:

- ✚ Las liberaciones de bajo riesgo son incrementales.
- ✚ Desacoplar despliegue y liberación.
- ✚ Concéntrate en reducir el tamaño del lote.
- ✚ Optimizar para la resiliencia.

A continuación se detallan un poco más las características de estos cuatro principios.

- ✚ **Los lanzamientos de bajo riesgo son incrementales.** El objetivo es diseñar los sistemas de tal manera que podamos lanzar cambios individuales (incluidos los cambios en la base de datos) de manera independiente, en lugar de tener que orquestar lanzamientos de big bang debido al alto acoplamiento entre múltiples sistemas diferentes. Esto normalmente requiere la creación de API's versionadas y a la implementación de patrones como el **interruptor de circuito automático (Circuit Breaker)**.

Es común que los sistemas de software realicen llamadas remotas al software que se ejecuta en diferentes procesos, probablemente en diferentes máquinas a través de una red. Una de las grandes diferencias entre las llamadas en memoria y las llamadas remotas es que las llamadas remotas pueden fallar o bloquearse sin respuesta hasta que se alcanza un límite de tiempo de espera. Lo que es peor, si tiene muchas llamadas a un proveedor que no responde, entonces puede quedarse sin recursos críticos que conducen a fallos en cascada en múltiples sistemas. Michael Nygard (en su libro Release It) popularizó el patrón del **interruptor de circuito** para evitar este tipo de cascada catastrófica.

La idea básica detrás del *interruptor de circuito* es muy simple. Envuelve una llamada de función protegida en un objeto *interruptor de circuito*, que supervisa los fallos. Una vez que los fallos alcanzan un cierto umbral, el *interruptor de circuito* se dispara y todas las demás llamadas al *interruptor de circuito* devuelven un error, sin que se realice la llamada protegida. Igualmente se monitoriza si el interruptor automático conmuta a activo.

La siguiente figura recoge el funcionamiento descrito.

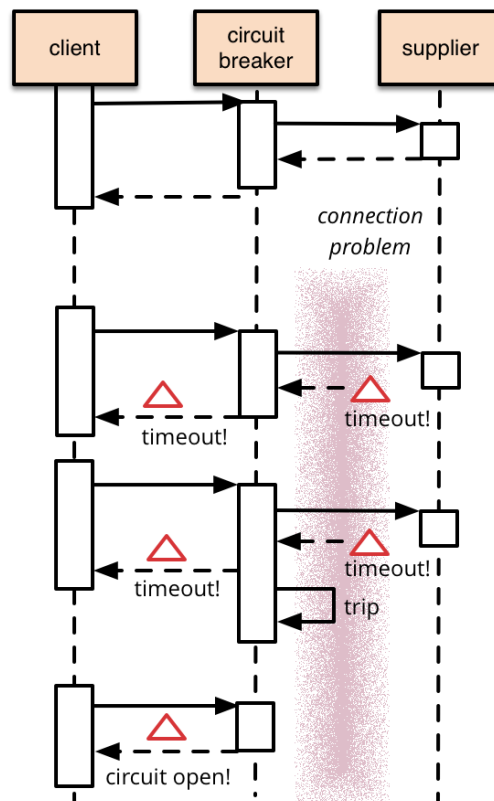


Figura 8 .- Ejemplo de implementación del patrón Circuit Breaker

Otro aspecto crítico para conseguir despliegues de bajo riesgo es controlar los cambios en la base de datos, que también se pueden implementar de forma incremental. Incluso organizaciones como Flickr, que despliegan varias veces al día, no implementan cambios en la base de datos con tanta frecuencia. En su lugar, utilizan el **patrón expand / contract**. La regla es que nunca cambias los objetos existentes todos a la vez. En su lugar, divide los cambios en pasos reversibles:

- ✚ Antes de que la nueva Release se libere, se agregan los nuevos objetos a la base de datos que serán requeridos por esa nueva versión.
- ✚ Se libera la nueva versión de la aplicación, que escribe en los nuevos objetos, pero lee de los objetos antiguos si es necesario para migrar los datos "perezosamente". Si se necesita revertir en este punto, se puede hacer sin tener que revertir los cambios de la base de datos.
- ✚ Finalmente, una vez que la nueva versión de la aplicación sea estable y sea seguro de que no se tendrá que retroceder, aplique el script del contrato para terminar de migrar los datos antiguos y eliminar los objetos antiguos.

- ✚ **Desacoplar despliegue y versión (Release).** La liberación de nuevas versiones del sistema no debería requerir tiempo de inactividad. Se puede utilizar un patrón llamado implementación azul-verde para permitir el tiempo de inactividad y la reversión en menos de un segundo, a pesar de que tomó decenas de minutos realizar la implementación. El objetivo final es separar la decisión técnica de desplegar de la decisión de negocios para lanzar una función, de modo que podamos implementar de forma continua pero lanzar nuevas funciones bajo demanda. Otros dos patrones de uso común que permiten este objetivo son el lanzamiento oscuro y la alternancia de funciones.

Usando el patrón de despliegue azul-verde, se despliega la nueva versión de la aplicación en paralelo con la versión anterior. Para pasar a la nueva versión, y volver a la versión anterior, cambiamos la configuración del balanceador como se ve en la siguiente figura siguiente.

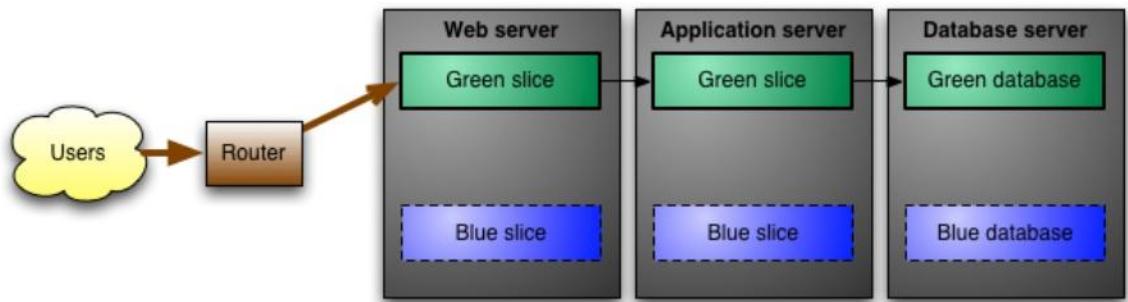


Figura 9 .- Ejemplo de implementación del patrón verde/azul

Una variante de la implementación azul-verde, aplicable cuando se ejecuta un grupo de servidores, es la liberación del canario (Canary Release¹). Con este patrón, en lugar de actualizar todo un clúster a la última versión de una vez, se hace de manera incremental. Por ejemplo, Facebook impulsa nuevas Builds a producción en tres fases como se ve en la siguiente figura:

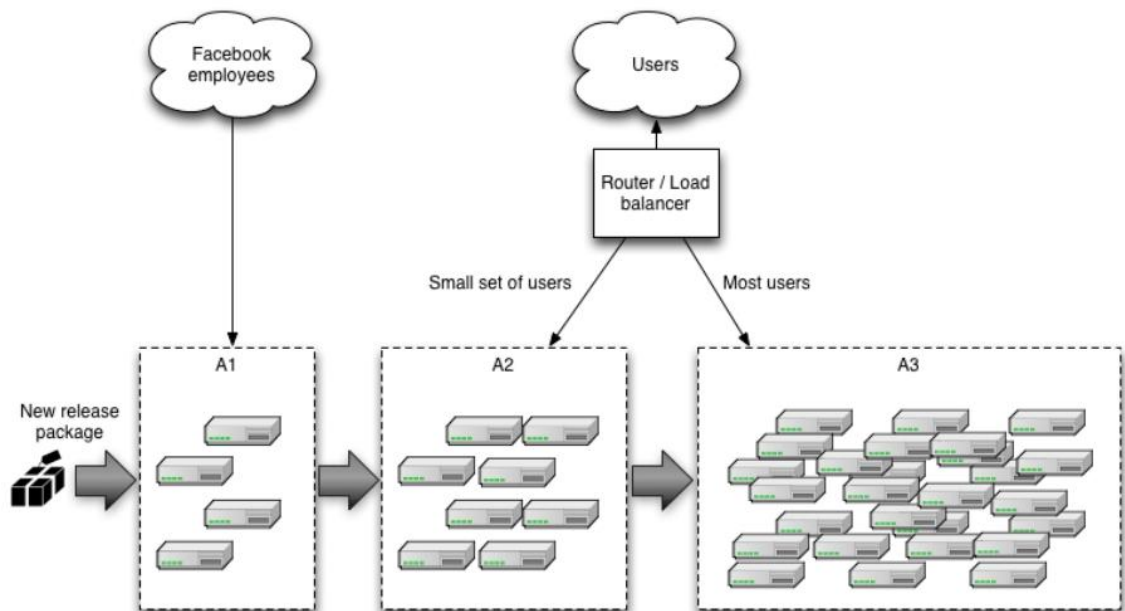


Figura 10 .- Ejemplo de implementación del patrón Canary Release

- 1) Primero, la compilación va a A1, un pequeño conjunto de cajas de producción a las que solo acceden los empleados.
- 2) Si el despliegue de A1 se ve bien, la compilación va a A2, un "par de miles" de cajas a las que solo se dirige un pequeño porcentaje de usuarios.
- 3) A1 y A2 son como los canarios en una mina de carbón: si se descubre un problema en estas etapas, la construcción no va más allá. Solo cuando no se producen problemas, la compilación se promueve a A3

¹ Se liberaban los canarios en una mina de carbón ya que eran más sensibles al gas grisú (Monóxido de carbono) y morían antes de que los mineros se vieran afectados.

- ✚ **Centrarse en reducir el tamaño del lote.** En contra de la intuición, la implementación en la producción con mayor frecuencia en realidad reduce el riesgo de lanzamiento cuando se realiza correctamente, simplemente porque la cantidad de cambio en cada implementación es menor. Cuando cada implementación consta de decenas de líneas de código o unos pocos ajustes de configuración, es mucho más fácil realizar el análisis de la causa raíz y restaurar el servicio en el caso de un incidente. Además, como se practica el proceso de implementación con tanta frecuencia, es obligatorio simplificarlo y automatizarlo, lo que reduce aún más el riesgo.
- ✚ **Optimizar para la resiliencia.** Los fallos son inevitables. Una vez aceptado se debe alejarse de la idea de invertir todo nuestro esfuerzo para prevenir o evitar problemas, y pensar en cómo restaurar el servicio lo más rápido posible cuando algo sale mal. Además, cuando ocurre un accidente, debe tratarse como una oportunidad de aprendizaje. Se sabe que los patrones descritos funcionan a gran escala en todo tipo de entornos, y demostrablemente aumentan el rendimiento mientras que al mismo tiempo aumentan la estabilidad en la producción. Sin embargo, la resiliencia no es solo una característica de los sistemas, es una característica de la cultura. Las organizaciones de alto rendimiento están trabajando constantemente para mejorar la capacidad de recuperación de sus sistemas al intentar romperlos e implementar las lecciones aprendidas en el curso de hacerlo, tal y como se ha planteado en el capítulo dedicado a la cultura de la organización y el ejército "simiano" de Netflix.

3.4. Entrega Continua y DevOps

El origen del término DevOps se debe al consultor belga, gerente de proyectos y practicante ágil Patrick Debois. Entre 2007-2008 tenía el encargo del gobierno belga para ayudar con las migraciones de los centros de datos. En particular, su papel fue en las pruebas de certificación / preparación. Sus tareas requerían que se relacionara con las actividades y relaciones entre los equipos de desarrollo de aplicaciones y los equipos de operaciones (servidor, red de base de datos). Sus experiencias, y frustraciones sobre los muros de separación y falta de cohesión entre los métodos de aplicación y los métodos de infraestructura, plantaron semillas de descontento para Debois. Su deseo por una mejor manera pronto lo llevaría a la acción.

En 2008, en la Conferencia Agile en Toronto, Andrew Schafer publicó una oferta para moderar una reunión para discutir el tema de "Infraestructura Agile" que solamente Patrick Debois atendió. Sus discusiones y el intercambio de ideas con otros avanzaron el concepto de "administración de sistemas ágiles", germen del grupo del mismo nombre que Debois y Shafer en Google, con un éxito limitado.

En 2009 durante la **Conferencia O'Reilly Velocity**, dos empleados de Flickr, John Allspaw, vicepresidente senior de operaciones técnicas, y Paul Hammond, director de ingeniería, dieron una presentación titulada "**10+ Despliegues por día: cooperación entre desarrolladores y operadores en Flickr**". La presentación tenía un estilo dramático, ya que Allspaw y Hammond interpretaron el papel de la controvertida interacción entre representantes de Desarrollo y Operaciones durante un despliegue de software típico, junto con todos los señalamientos / culpas que siguen, como "¡No es mi código, son sus máquinas!". Su presentación demostró que la única forma racional de avanzar es que el desarrollo de aplicaciones y las actividades de operaciones sean sin fisuras, transparentes y totalmente integradas. Con el tiempo, esta presentación ha alcanzado un estatus legendario, y se ha visto históricamente como el momento seminal en el tiempo en el que se empujó a la industria de TI a apostar por métodos y prácticas que ahora conocemos como DevOps.

Desafortunadamente, Debois tuvo que ver la presentación de Allspaw / Hammond por Internet. Se inspiró y a instancias de otros a través de las redes sociales, celebró su propia conferencia llamada **Devopsdays** en Gante, Bélgica. Este es el momento en el tiempo en que el término "**DevOps**" había aterrizado oficialmente en los libros de historia.

DevOps engloba de manera natural y obligada los conceptos de Integración, entrega y despliegue continuo, yendo más allá en está la relación de manera natural con el mundo del desarrollo. Para definirlo, habría que considerar DevOps como una cultura o “modo de ser y comportarse” o como los más convencidos que hacen referencia el como un “viaje”.

Los conceptos de esta cultura se recogen en los siguientes estados, que en sí mismo son los objetivos de mejora a conseguir:

- ✚ Construir la base
- ✚ Normalizar la pila de tecnología
- ✚ Estandarizar y reducir la variabilidad.
- ✚ Expandir las prácticas de DevOps
- ✚ Automatizar la entrega de infraestructura
- ✚ Proporcionar capacidades de autoservicio

Los principios y valores de DevOps están recogidos en el Tema 37, Metodologías ágiles para la gestión de proyectos. Metodologías lean, apartado 5. Para más información se puede consultar la bibliografía referenciada.

3.5. Herramientas de Integración Continua

Una vez analizado el proceso de Integración y entrega continua, se van a analizar distintas herramientas que facilitan la automatización del proceso. La elección entre ellas depende de múltiples factores y la lista crece y evoluciona constantemente.

En la imagen siguiente se puede ver en forma de tabla periódica un resumen del sector hecho por XebiaLabs, uno de los fabricantes líderes del mercado. XebiaLabs también ofrece la posibilidad de crear tu propio Pipeline incorporando los distintos productos que utilice una organización. Esta imagen resumen perfectamente las categorías de herramientas y un variado conjunto de opciones tecnológicas que van desde OpenSource a Enterprise.

PERIODIC TABLE OF DEVOPS TOOLS (V3)																																																																																																																							
Open Source		Source Control Mgmt.		Deployment		Analytics																																																																																																																	
Free		Database Automation		Containers		Monitoring																																																																																																																	
Freemium		Continuous Integration		Release Orchestration		Security																																																																																																																	
Paid		Testing		Cloud		Collaboration																																																																																																																	
Enterprise		Configuration		AI/Ops																																																																																																																			
1 GI GitLab	2 Sp Splunk	3 Gh GitHub	4 Dt Datadog	5 XLR XebiaLabs XL Release	6 Aws AWS	7 Az Azure	8 Gc Google Cloud	9 Op OpenShift	10 Sg Sumo Logic	11 Sv Subversion	12 Db DBMaestro	13 Dk Docker	14 Ur UrbanCode Release	15 Af Azure Functions	16 Ld Lambda	17 Ic IBM Cloud	18 Fd F5	19 Cw ISPW	20 Dp Delphix	21 Jn Jenkins	22 Cs Codeship	23 Fn FitNesse	24 Ju JUnit	25 Ka Karma	26 Su SoapUI	27 Ch Chef	28 Tf Terraform	29 XLD XebiaLabs XL Deploy	30 Ud UrbanCode Deploy	31 Ku Kubernetes	32 Cc CA CD Director	33 Pr Plutora Release	34 Al Alibaba Cloud	35 Os OpenStack	36 Ps Prometheus	37 At Artifactory	38 Rg Redgate	39 Ba Bamboo	40 Vs VSTS	41 Se Selenium	42 Jm JMeter	43 Ja Jasmine	44 Sl Sauce Labs	45 An Ansible	46 Ru Rudder	47 Oc Octopus Deploy	48 Go GoCD	49 Ms Mesos	50 Gke GKE	51 Om OpenMake	52 Cp AWS CodePipeline	53 Cy Cloud Foundry	54 It ITRS	55 Nx Nexus	56 Fw Flyway	57 Tr Travis CI	58 Tc TeamCity	59 Ga Gatling	60 Tn TestNG	61 Tt Tricentis Tosca	62 Pe Perfecto	63 Pu Puppet	64 Pa Packer	65 Cd AWS CodeDeploy	66 Ec ElectricCloud	67 Ra Rancher	68 Aks AKS	69 Rk Rkt	70 Sp Spinnaker	71 Ir Iron.io	72 Mg Microsoft	73 Bb BitBucket	74 Pf Perforce	75 Cr Circle CI	76 Cb AWS CodeBuild	77 Cu Cucumber	78 Mc Mocha	79 Lo Locust.io	80 Mf Micro Focus UFT	81 Sa Salt	82 Ce CEngine	83 Eb ElasticBox	84 Ca CA Automate	85 De Docker Enterprise	86 Ae AWS ECS	87 Cf Codefresh	88 Hm Helm	89 Aw Apache OpenWhisk	90 Ls Logstash	91 XLI XebiaLabs XL Impact	92 Ki Kibana	93 Nr New Relic	94 Dt Dynatrace	95 Dd Datadog	96 Ad AppDynamics	97 El ElasticSearch	98 Ni Nagios	99 Zb Zabbix	100 Zn Zenoss	101 Cx Checkmarx SAST	102 Sg Signal Sciences	103 Bd BlackDuck	104 Sr SonarQube	105 Hv HashiCorp Vault	106 Sw ServiceNow	107 Jr Jira	108 Tl Trello	109 Sk Slack	110 St Stride	111 Cn CollabNet VersionOne	112 Ry Remedy	113 Ac Agile Central	114 Og OpsGenie	115 Pd Pagerduty	116 Sn Snort	117 Tw Tripwire	118 Ck CyberArk	119 Vc Veracode	120 Ff Fortify SCA

**XebiaLabs**
Enterprise DevOps

 Follow @xebialabs

Figura 11 .- Tabla periódica de productos de DevOps (CI/CD) de XebiaLabs

3.5.1. Herramientas de control de versiones

Como se comentó en el apartado relativo al control de versiones, herramientas de este tipo hay muchas y variadas, siendo la lista muy extensa al ser una herramienta básica. Ejemplos de este tipo de herramientas son entre otros: **CVS**, **Subversion**, SourceSafe, ClearCase, Darcs, Bazaar, Plastic SCM, **Git**, SCCS, **Mercurial**, Perforce, Fossil SCM, **Team Foundation Server** o BitBucket.

Para un detalle razonable de las funcionalidades generales de un sistema de control de versiones puede consultar: https://es.wikipedia.org/wiki/Control_de_versiones o la versión inglesa más completa https://en.wikipedia.org/wiki/Version_control

3.5.1.1. GitHub

GIT es un sistema de control de administración de código fuente diseñado tanto para proyectos grandes como pequeños. Git se puede utilizar en “on-site” o de manera remota alojado en un servidor externo, siendo este uno de sus puntos fuertes. Muchas organizaciones eligen hacer ambas cosas, almacenar algunos proyectos en las instalaciones y otros dentro del servicio online.

-  Como servidor Git remoto, GitHub proporciona servicios gratuitos y premium.
-  Cuenta con una comunidad de más de 26 millones desarrolladores y 72 millones de repositorios en todo el mundo.
-  GitHub es un estándar de facto. Muchos otros repositorios soportan la integración y formatos de Git.
-  Una ventaja significativa al usar GitHub es su base de usuarios relativamente grande y casi ubicua, dando como resultado la disponibilidad de muchas integraciones.

3.5.1.2. BitBucket

BitBucket es el servicio de control de versiones colaborativo alojado de Atlassian para proyectos que utilizan Git o Mercurial. Como parte de la pila de Atlassian, BitBucket ofrece integración con productos de Atlassian como Jira y Trello. El mercado de Atlassian también permite una amplia variedad de integraciones. Aunque pueden encontrar que BitBucket no tiene tantas integraciones como GitHub, es probable que los desarrolladores encuentren la integración que necesitan.

Las fortalezas de BitBucket incluyen escalabilidad superior y colaboración en equipo. Las recientes adiciones a BitBucket amplían las capacidades básicas del repositorio y se orientan hacia la integración continua y otras áreas funcionales. Un ejemplo de mayor funcionalidad es la integración de BitBucket Pipelines para admitir la integración continua y la entrega continua.

3.5.1.3. Subversion (SVN)

Subversion (SVN) es un sistema de control de versiones, basado en software libre con licencia Apache/BSD.

Es flexible, permite crear diferentes repositorios con usuarios propios en cada caso o usuarios que pueden acceder a varios repositorios.

Realiza un histórico y seguimiento de ficheros y directorios aun siendo copiados o renombrados.

Permite medir de forma tangible el trabajo realizado por cada usuario a diario.

Las modificaciones se pueden realizar de forma atómica (por ejemplo, solucionar un bug que afecta a 10 ficheros se puede considerar un único cambio).

Se integra fácilmente con otros productos como gestores de incidencias (Mantis BT o JIRA) o sistemas de integración continua (Apache Continuum, Jenkins o CruiseControl).

3.5.2. Herramientas de Integración Continua

La siguiente tabla compara las características del software de integración continua más populares respecto a su compatibilidad con algunos de los CMS/CVS más conocidos. Como ya se ha comentado en apartados anteriores y que son base para un sistema de entrega continua.

Name	ClearCase	CVS	Darcs	Git	Mercurial	Subversion	Team Foundation Server
Apache Gump	No	Yes	No	No	No	Yes	No
AppVeyor	No	No	No	Yes	Yes	Yes ^[20]	No
Bamboo	Yes	Yes	No	Yes	Yes	Yes	Yes ^[22]
Buddy	No	No	No	Yes	No	No	No
BuildBot	No	Yes	Yes	Yes	Yes	Yes	No
BuildMaster	Yes	Yes	No	Yes	Yes	Yes	Yes
CABIE	No	Yes	No	No	No	Yes	No
CruiseControl	Yes	Yes	Yes	Yes	Yes	Yes	Yes
CruiseControl.NET	Yes	Yes	No	Yes	Yes	Yes	Yes
Distelli	No	No	No	Yes	Yes	No	No
Jenkins	Yes	Yes	Yes ^[23]	Yes	Yes	Yes	Yes
OpenMake Software Meister	Yes	Yes	No	Yes	No	Yes	Yes
QuickBuild (software)	Yes	Yes	No	Yes	Yes	Yes	Yes
Semaphore (software)	No	No	No	Yes	No	No	No
TeamCity	Yes	Yes	No	Yes	Yes	Yes	Yes
Team Foundation Server	No	No	No	Yes	No	Yes	Yes
Vexor	No	No	No	Yes	No	No	No

Figura 12.- Comparativa de compatibilidad de herramientas de Integración Continua Vs Control de versiones de

Aunque la Integración continua es una práctica que no requiere herramientas particulares para implementar, es útil (indispensable) usar un servidor de Integración continua. Uno de los primeros servidores de IC es CruiseControl, una herramienta de código abierto originalmente creada por ThoughtWorks y mantenida por una amplia comunidad. Desde entonces, han aparecido otros servidores de IC, tanto de código abierto como comercial.

Si bien CruiseControl era una herramienta relativamente simple, representaba un compromiso con el software y la integridad cronológica. Pronto, otros contendientes surgieron alrededor del concepto. Hudson, otra herramienta de CI escrita en Java, se lanzó en 2005, pero en realidad no comenzó a superar a CruiseControl hasta 2008.

El autor de Hudson, Kohsuke Kawaguchi, había estado trabajando en Sun Microsystems, que Oracle compró en 2010. Al igual que CruiseControl, Hudson era libre y de código abierto, por lo que cuando Oracle anunció sus planes de marca y comercialización, la comunidad de Hudson abandonó la escena en 2011 y reapareció bajo el nuevo nombre de Jenkins, siendo la herramienta más utilizada como motor de integración continua.

Entre las herramientas de Integración continua destacan además de las recogidas en la tabla anterior, CircleCI, GitLab, Travis CI, Atlassian Bamboo siendo Jenkins siempre referencia obligada.

Los servidores de integración continua (IC o CI en inglés) son el punto de entrada para muchas nuevas organizaciones que abrazan el “camino” DevOps como una extensión del desarrollo ágil. Los servidores permiten la construcción y prueba automatizadas, junto con diversos niveles de notificaciones y avisos fundamentales para mantener los esfuerzos ágiles en pista.

El proyecto de código abierto Jenkins® rastrea las instalaciones activas, por ejemplo, y a principios de 2018 se estimaban en torno a 250,000 Jenkins activos, siendo de lejos el servidor de CI / CD más dominante en términos de cuota de mercado.

Como suele pasar en muchas disciplinas, las herramientas que surgen para cubrir una necesidad suelen ampliar sus capacidades y funcionalidades, en este caso muchas herramientas de CI han agregado capacidades de entrega continua (CD). Por ese motivo las diferencias entre la integración continua, Entrega continua y Liberación de aplicaciones automatizada (ARA - Application Release Automation) se vuelven más difusas.

Y por si no fuera poco, el término evoluciona, ARA a veces se nombra como Application Release Orchestration (ARO) o Continuous Delivery and Release Automation (CDRA).

Entre las herramientas de automatización de releases nos encontramos XebiaLabs XL Deploy & Release, Urbancode Deploy & Release, Octopus Deploy, AWS CodeDeploy, ElasticBox, CA Automtic, ElectricCloud, GoCD.

3.5.2.1. Jenkins

Si hay una herramienta que la mayoría de los equipos de DevOps tienen desplegado, es Jenkins. Hay otras herramientas, pero Jenkins es la "Got DevOps?" de referencia. Si Jenkins tiene un problema, es uno de abundancia, en el tema de plugin es “ostentoso”, hasta 1.500 complementos diferentes están disponibles para Jenkins gracias a su comunidad lo que permite virtualmente integrarse con todo.

Ciertamente, Jenkins ha tomado medidas para hacer la selección de complementos más fácil para sus usuarios. Siempre hay que recordar que cuanto menos, mejor y ya se irán incorporando cuando se necesiten realmente. Lo básico sería una gestión de código fuente (SCM) como GitHub, BitBucket, SVN o Local Git, y otro plugin para compilaciones (Gradle o Maven, por ejemplo).

Jenkins 2.0 también incluye Jenkins Pipeline y una versión más amigable gracias al interfaz (ocean blue).

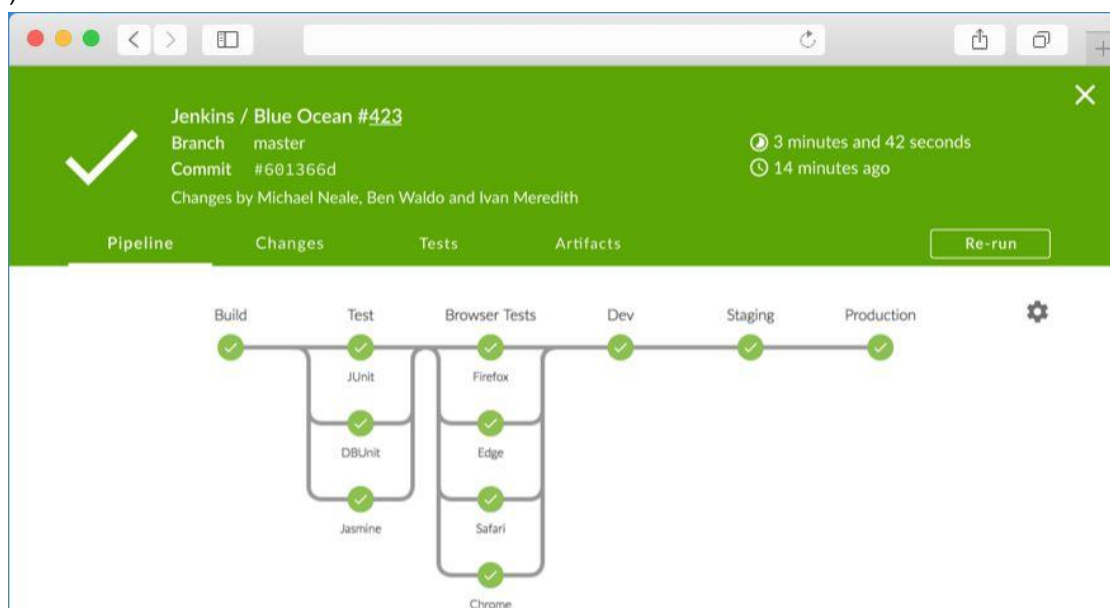


Figura 13 .- Jenkins 2.0 Pipeline interfaz ocean blue

El 12 de marzo de 2019 La Linux Foundation, junto con CloudBees, Google y otras compañías, lanzaron una nueva base de software de fuente abierta llamada Continuous Delivery Foundation (CDF). *“La CDF cree en el poder de la entrega continua, y tiene como objetivo fomentar y mantener el ecosistema de código abierto, proyectos neutrales a los proveedores al que los colaboradores de Jenkins han decidido unirse.*

CloudBees es un activo colaborador de la comunidad Jenkins y ofrece servicios de soporte empresarial para Entrega Continua.

3.5.3. Repositorios de Artefactos

3.5.3.1. Artifactory – Jfrog

Es un repositorio centralizado y estable de librerías J2EE y recursos compartidos.

Permite armonizar la gestión y administración de paquetes y librerías actuando de intermediario entre los Maven de desarrollo / integración y los repositorios públicos externos disponibles a lo largo y ancho de la Internet. Mantiene una caché local de repositorios descargados, conectándose a Internet solo cuando sea necesario.

Altamente configurable, habilita el mantenimiento de solamente aquellos paquetes y versiones aprobadas por la organización de informática para desarrollo J2EE, e impidiendo el acceso a productos o librerías no aceptados.

3.5.3.2. Apache Archiva

Apache Archiva es un software de administración de repositorios extensible que ayuda a cuidar su propio repositorio de artefactos de compilación personal o empresarial. Es el compañero perfecto para herramientas de construcción como Maven, Continuum y ANT.

3.5.4. Empaquetado y despliegue de aplicaciones

3.5.4.1. Apache Maven

Es una herramienta de software para la gestión y construcción de proyectos Java basada en un modelo de configuración de construcción simple mediante fichero de Modelo de Objetos de Proyecto (POM) que se define y mantiene en un fichero XML, en el cual se definen las dependencias de otros módulos y componentes externos, así como el orden de construcción de los elementos.

Maven funciona en red: el motor incluido en su núcleo puede dinámicamente descargar tanto plugins como librerías desde un repositorio remoto.

Incluye de serie plugins para la mayoría de tareas requeridas: comprobación de estilos, compilación, pruebas unitarias, empaquetado, y despliegue en servidores remotos. Controla librerías, y sus dependencias.

-  Maven Cargo: es un plugin que permite manipular servidores de aplicaciones de forma estándar, arrancandolos, parandolos e instalando aplicaciones. Maven Cargo tiene soporte para múltiples servidores de aplicaciones, Jboss, Tomcat, ORACLE Weblogic, etc...
-  Maven Surefire: Es un Framework de test y se utiliza durante el ciclo de vida de pruebas, para ejecutar las pruebas y crear informes, en formato de texto, xml o html, en cualquier caso estos informes son interpretados por el sistema de integración continua. Maven Surefire puede utilizar diferentes herramientas para la gestión de las pruebas, el ejemplo más extendido es JUnit.

Ant es otra herramienta más antigua que MAVEN.

3.5.4.2. Gradle

Gradle es un sistema de automatización de construcción de código abierto que construye sobre los conceptos de Apache Ant y Apache Maven e introduce un lenguaje específico del dominio (DSL) basado en Groovy en vez de la forma XML utilizada por Apache Maven para declarar la configuración

de proyecto. Gradle utiliza un grafo acíclico dirigido ("DAG") para determinar el orden en el que las tareas pueden ser ejecutadas.

Gradle fue diseñado para construcciones multi-proyecto las cuales pueden crecer para ser bastante grandes, y da apoyo a construcciones incrementales determinando inteligentemente qué partes del árbol de construcción están actualizadas, de modo que cualquier tarea dependiente a aquellas partes no necesitara ser reejecutada.

Los plugins iniciales están principalmente centrados en el desarrollo y despliegue en Java, Groovy y Scala, pero existen más lenguajes y workflows de proyecto en el roadmap.

3.5.5. Herramientas de Testing

La variedad de herramientas de pruebas es muy grande, cubriendo tanto aspectos no funcionales como el rendimiento en el caso de JMeter, o aspectos funcionales desde el nivel unitario, hasta pruebas de aceptación.

A continuación se recogen algunas de ellas, tanto de uso libre, freemium o con diversas modalidades de licenciamiento: FitNesse, **Junit**, Karma, SoapUI, Selenium, Jmeter, Jasmine, Sauce Labs, Gatling, TestNG, Tricentis Tosca, Perfecto, Cucumber, Mocha, Locust.io, MicroFocus UFT que adquirió la división de SW de HP incluyendo LOAD-Runner.

3.5.5.1. JMeter

JMeter es un proyecto de Apache que puede ser utilizado como una herramienta para realizar pruebas de carga para analizar y medir el desempeño de distintos servicios y aplicaciones, con énfasis en el entorno web.

JMeter puede ser usado como una herramienta de pruebas unitarias para conexiones de bases de datos con JDBC, FTP, LDAP, Servicios web, JMS, HTTP y conexiones TCP genéricas. También puede ser configurado como un monitor (solución ad-hoc).

A veces se clasifica JMeter como herramienta de "generación de carga", pero esto no es una descripción completa de la herramienta. JMeter soporta aserciones para asegurar que los datos recibidos son correctos, por lo que es una herramienta que permite la realización de pruebas automáticas.

3.5.5.2. Junit – [X]unit

JUnit es un conjunto de bibliotecas creadas por Erich Gamma y Kent Beck que son utilizadas en programación para hacer pruebas unitarias de aplicaciones Java, permite realizar la ejecución de clases de manera controlada, para poder evaluar si el funcionamiento de cada uno de los métodos de la clase se comporta como se espera.

Es decir, en función de algún valor de entrada se evalúa el valor de retorno esperado; si la clase cumple con la especificación, entonces JUnit devolverá que el método de la clase pasó exitosamente la prueba; en caso de que el valor esperado sea diferente al que regresó el método durante la ejecución, JUnit devolverá un fallo en el método correspondiente.

JUnit es también un medio de controlar las pruebas de regresión, necesarias cuando una parte del código ha sido modificado y se desea ver que el nuevo código cumple con los requerimientos anteriores y que no se ha alterado su funcionalidad después de la nueva modificación.

Las herramientas de desarrollo como Eclipse cuentan con plug-ins que permiten que la generación de las plantillas necesarias para la creación de las pruebas de una clase Java se realice de manera automática, facilitando al programador enfocarse en la prueba y el resultado esperado, y dejando a la herramienta la creación de las clases que permiten coordinar las pruebas.

3.5.5.3. SoapUI

SoapUI es una herramienta, desarrollada en java (2005 SourceForge), para la realización de pruebas a aplicaciones con arquitectura orientada a servicio (SOA) y *transferencia de estado representacional* (REST). Soporta múltiples protocolos como SOAP, REST, HTTP, JMS, AMF y JDBC. Posee una versión de Código abierto y otra versión de pago realizada por la compañía SmartBear.

3.5.5.4. Selenium

Selenium es un entorno de pruebas de software para aplicaciones basadas en la web. Selenium provee una herramienta de grabar/reproducir para crear pruebas sin usar un lenguaje de scripting para pruebas (Selenium IDE). Incluye también un lenguaje específico de dominio para pruebas (Selenese) para escribir pruebas en un amplio número de lenguajes de programación populares incluyendo Java, C#, Ruby, Groovy, Perl, Php y Python. Las pruebas pueden ejecutarse entonces usando la mayoría de los navegadores web modernos en diferentes sistemas operativos como Windows, Linux y OSX.

Selenium está formado por los siguientes componentes:

- ✚ **Selenium IDE** es un entorno de desarrollo integrado para pruebas con Selenium. Está implementado como una extensión de Firefox y permite grabar, editar y depurar pruebas. Originalmente se le conoció como Selenium Recorder.
Se pueden desarrollar automáticamente scripts al crear una grabación y de esa manera se puede editar manualmente con sentencias y comandos para que la reproducción de nuestra grabación sea correcta
Los scripts se generan en Selenese, un lenguaje de scripting especial para Selenium. Selenese provee comandos que ejecutan acciones sobre objetos en el navegador, como hacer click en un enlace, seleccionar de una lista de opciones, verificar la presencia de un texto en particular así como para tomar la totalidad de la página producto de las acciones.
Características:
 - ✓ Grabación y reproducción fácil
 - ✓ Selección inteligente de campos usando ID, nombre o XPath según se necesite.
 - ✓ Autocompletado de los comandos de Selenium más comunes.
 - ✓ Pruebas de revisión cruzada
 - ✓ Depuración y puntos de verificación (breakpoint)
 - ✓ Almacenar las pruebas como Selenese, Ruby, Java y otros formatos.
 - ✓ Soporte al archivo user-extensions.js
 - ✓ Opción para asertar el título de la página.
 - ✓ Opción de modificarle a la medida con el uso de complementos
- ✚ **Selenium Client API**: Interfaz de programación de aplicaciones (API) de clientes Como alternativa a escribir pruebas en Selenese, las pruebas pueden escribirse en varios lenguajes de programación, éstos se comunican con Selenium mediante llamadas a los métodos de Selenium Client API. Actualmente Selenium provee API para Java, C#, Ruby y Python. Con Selenium 2 se presentó una nueva API de clientes, con WebDriver como componente central, aunque la anterior API puede seguirse usando llamando a la clase Selenium.
- ✚ **Selenium Remote Control (RC)** es un servidor escrito en Java que acepta comandos al navegador vía HTTP. RC hace posible escribir pruebas automatizadas para aplicaciones web, en cualquier lenguaje de programación lo que permite una mejor integración de Selenium a entornos de prueba existentes.
- ✚ **Selenium WebDriver** es el sucesor de Selenium Remote Control. Selenium WebDriver acepta comandos (enviados en Selenese o vía el API de cliente) y los envía a un navegador. Esto se implementa a través de un controlador del navegador específico para cada navegador que envía los comandos y trae los resultados de regreso.
- ✚ **Selenium Grid** es un servidor que permite usar instancias de navegador ejecutándose en máquinas remotas y permite ejecutar pruebas en paralelo en múltiples máquinas y manejar diferentes versiones y configuraciones de manera centralizada.

3.5.6. Herramientas de control de Calidad Estático

La calidad de código suele decirse que es un atributo interno de calidad, dado que no se hace visible al usuario. Pero llega un momento en el cual este atributo de calidad pasa de ser interno a externo, y esto se da cuando el hecho de tener modificar el código para hacer un cambio lleva mucho más tiempo del que debería. Con el fin de verificar la calidad interna de un sistema se suelen hacer análisis de código con SonarQube o herramientas similares como **Checkstyle, PMD o FindBugs**.

3.5.6.1. SonarQube

SonarQube, como tantas otras herramientas similares, permite realizar análisis estático de código fuente de manera automática, buscando patrones con errores, malas prácticas o incidentes. Además, realiza un cálculo de la deuda técnica. Dentro de las verificaciones que hacen herramientas como SonarQube, se encuentran las siguientes:

-  Detección de código duplicado.
-  Falta de pruebas unitarias, falta de comentarios.
-  Código spaghetti, complejidad ciclomática, alto acoplamiento.
-  Tamaño de archivos de código.
-  Tamaño de métodos.
-  No adecuación a estándares y convenciones de código.
-  Vulnerabilidades conocidas de seguridad.

A través de definir “Quality Gates” es posible marcar los umbrales aceptables en el proceso de desarrollo, y estos pueden ser definidos en base a distintas métricas. Esto es muy útil si se integra SonarQube en el pipeline de CI/CD.

Un *Quality Gate* puede ser definido para responder preguntas como:

-  No hay issues nuevos que sean críticos o bloqueantes.
-  El code coverage de las pruebas unitarias del nuevo código es mayor a 85%

Para poder utilizar SonarQube se necesita instalar un componente servidor, donde se encuentra el engine que realiza el análisis y almacena los resultados, y se debe invocar de alguna manera el análisis, lo cual puede realizarse con un cliente llamado **SonarQube Scanner** o con un plugin de Maven. También se puede integrar el análisis al IDE que se esté utilizando, con un plugin llamado SonarLint.

La imagen siguiente muestra el aspecto que tendría SonarQB, donde se muestran líneas de código, Número de clases, comentarios y duplicados en la parte superior izquierda, En la superior derecha el porcentaje de cumplimiento de criterios “Quality gates” y los incumplimientos clasificados de mayor importancia (Bloqueantes) a menor (Info). Debajo se muestra la complejidad, la mantenibilidad, y en la parte inferior derecha la “Deuda Técnica”

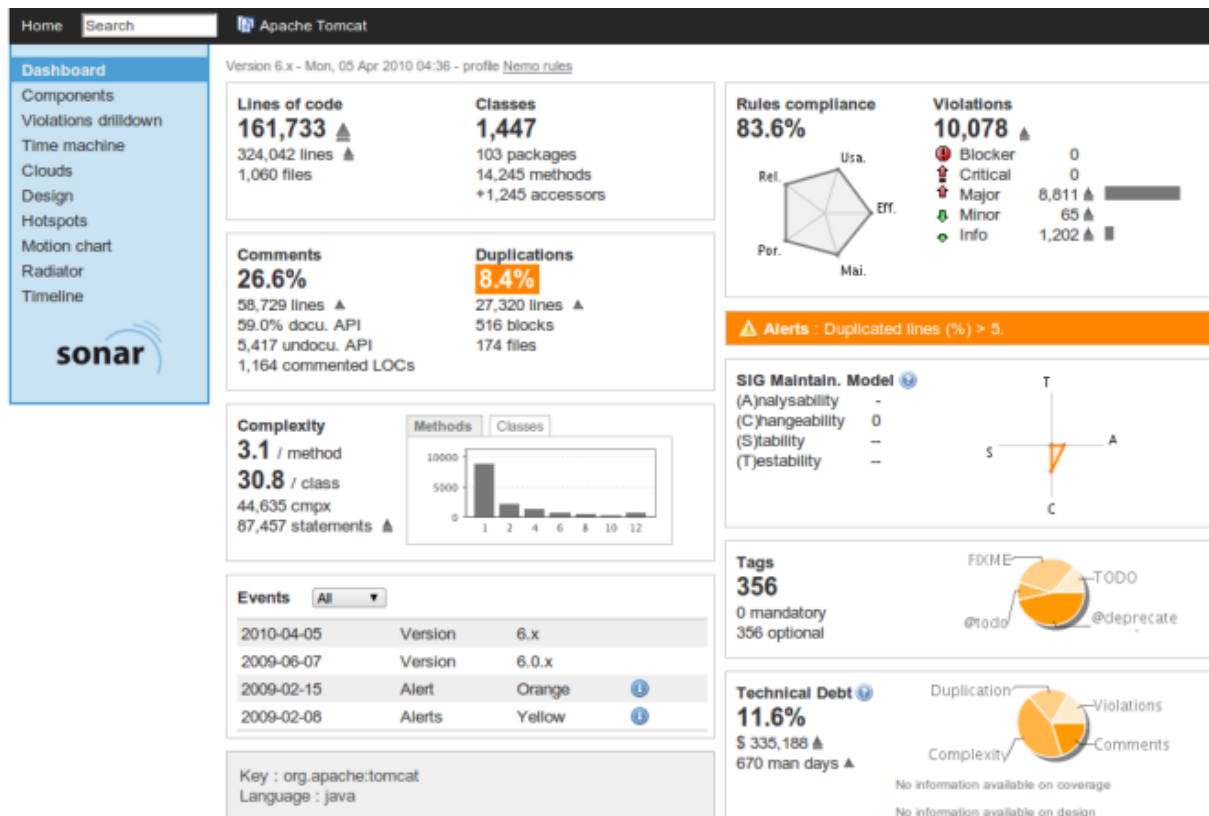


Figura 14 .- Consola SonarQube

3.5.7. Herramientas de gestión de requisitos y casos de pruebas

Mediante herramientas de gestión de requisitos y casos de pruebas podemos gestionar los casos de pruebas y requisitos asociados a los proyectos desde un mismo interfaz de manera sencilla, permitiendo organizar las pruebas asociadas a los proyectos y requisitos definidos.

3.5.7.1. TestLink

TestLink nos ofrece la posibilidad de gestionar los casos de pruebas y requisitos asociados desde un mismo interfaz, permitiendo asignar roles y permisos independientes a los integrantes del equipo de pruebas para cada una de las secciones y planes de pruebas creados. Mediante el interfaz de comunicación XML-RPC podemos conectarnos con aplicaciones de ejecución de pruebas para automatizar estas y recoger los resultados para mostrarlos en la herramienta, aunque es necesario realizar desarrollos para la integración con herramientas de ejecución de pruebas e integración continua, existen plugins desarrollados para la integración con sistemas como Jenkins, Selenium, etc.

4. RESUMEN ESQUEMÁTICO

Introducción - Integración continua

Modelo informático propuesto inicialmente por Martin Fowler que consiste en hacer integraciones automáticas de un proyecto lo más frecuente posible para poder detectar fallos cuanto antes.

Entrega continua

La entrega continua es la capacidad de obtener cambios de todo tipo, incluidas nuevas funciones, cambios de configuración, correcciones de errores y experimentos, en producción o en manos de los usuarios, de forma segura y rápida de una manera sostenible.

La entrega continua nos ayuda a lograr varios objetivos importantes:

- ✓ Liberaciones de bajo riesgo.
- ✓ Time-to-market (plazos de comercialización) más rápidos.
- ✓ Mejor calidad.
- ✓ Costos más bajos.
- ✓ Mejores productos.
- ✓ Equipos más felices.

Principios de la entrega continua

- ✓ Incorporar la calidad intrínsecamente en el proceso de construcción.
- ✓ Trabajar en pequeños lotes.
- ✓ Los ordenadores realizan tareas repetitivas, las personas resuelven problemas.
- ✓ Continuamente persigue la mejora continua.
- ✓ Todos son responsables.

La reducción en el rendimiento general de la organización, a menudo se debe a problemas sistémicos como:

- ✓ Sistemas de gestión deficientes.
- ✓ Ciclos de presupuesto anual.
- ✓ Incentivos que recompensan los comportamientos incorrectos o no óptimos.

Fundamentos de entrega continua

Se basa en tres pilares o fundamentos:

- ✓ Gestión de configuración integral.
- ✓ Integración continua.
- ✓ Pruebas continuas.

Gestión de la configuración

- ✓ La Gestión de Configuración de Software trata y controla:
- ✓ La elaboración de código fuente por varios desarrolladores simultáneamente.
- ✓ El seguimiento del estado de las fases del desarrollo (versiones) y sus cambios (control de versiones).
- ✓ La conducción de la integración de las partes del software en un solo producto de software.

Integración continua

Prácticas clave para una integración continua efectiva son:

- ✓ Mantener un repositorio de código único.
- ✓ Automatizar la construcción.
- ✓ Build auto-probable.
- ✓ Commit con la línea principal todos los días.
- ✓ Cada COMMIT debe construir la línea principal en una máquina de integración.
- ✓ Arreglar las construcciones rotas inmediatamente.
- ✓ Mantener el proceso de construcción rápido.
- ✓ Probar en un clon del entorno de producción.
- ✓ Hacer fácil para cualquiera obtener el último ejecutable.

- ✓ Todos pueden ver lo que está pasando.

Testing continuo

La realización de "Pruebas automáticas" implica la automatización de un proceso manual, es decir la prueba automatizada se utiliza para ejecutar recurrentemente, rápida y repetida los escenarios de prueba que se realizaron de forma manual.

Implementación de la entrega continua

- ✓ Arquitectura
En una arquitectura empresarial habitualmente preocupan diferentes atributos como la disponibilidad, la seguridad, el rendimiento, la facilidad de uso, etc. En la entrega continua, se introducen dos nuevos atributos arquitectónicos: capacidad de prueba y de despliegue.
- ✓ Cultura
La cultura corporativa constituye el conjunto de normas, valores y pautas de conducta que comparten los integrantes de la empresa, que se ve reflejado en sus comportamientos.
- ✓ Patrones
Resultan necesarios y recomendables para implementar la entrega continua:
 - El canal de despliegue
 - Patrones para lanzamientos de bajo riesgo.

Entrega Continua y DevOps

- ✚ El origen del término DevOps se debe al consultor belga, gerente de proyectos y practicante ágil **Patrick Debois**.
- ✚ En 2008, en la Conferencia Agile en Toronto, Andrew Schafer publicó una oferta para moderar una reunión para discutir el tema de "**Infraestructura Agile**". Surge el concepto de "**administración de sistemas ágiles**".
- ✚ 2009, John Allspaw, vicepresidente senior de operaciones técnicas, y Paul Hammond, director de ingeniería, dieron una presentación titulada "10+ Despliegues por día: cooperación entre desarrolladores y operadores en Flickr".
- ✚ 2009 Debois celebra su propia conferencia **Devopsdays** en Gante, Bélgica. El término "**DevOps**" había aterrizado oficialmente en los libros de historia.

Los elementos más significativos de DevOps son:

- ✚ Es la única forma racional de avanzar es que el desarrollo de aplicaciones y las actividades de operaciones sean sin fisuras, transparentes y totalmente integradas.
- ✚ DevOps engloba de manera natural y obligada los conceptos de Integración, entrega y despliegue continuo, yendo más allá en está la relación de manera natural con el mundo del desarrollo.
- ✚ Para definirlo, habría que considerar DevOps como una cultura o "modo de ser y comportarse"

Los conceptos de la cultura DevOps (objetivos de mejora a conseguir) se recogen en los siguientes estados:

- ✓ Construir la base
- ✓ Normalizar la pila de tecnología
- ✓ Estandarizar y reducir la variabilidad.
- ✓ Expandir las prácticas de DevOps
- ✓ Automatizar la entrega de infraestructura
- ✓ Proporcionar capacidades de autoservicio

Herramientas de Integración Continua

Herramientas de control de versiones

- ✓ GITHUB
- ✓ BITBUCKET
- ✓ SUBVERSION (SVN)

Herramientas de integración continua

- ✓ JENKINS

Repositorios de artefactos

- ✓ ARTIFACTORY – JFROG
- ✓ Apache Archiva
- ✓ Nexus Sonatype

Empaquetado y despliegue de aplicaciones

- ✓ APACHE MAVEN
- ✓ GRADLE

Herramientas de testing

- ✓ JUNIT – [X]UNIT
- ✓ SELENIUM

Herramientas de control de calidad estático

- ✓ SONARQUBE

Herramientas de gestión de requisitos y casos de pruebas

- ✓ Testlink

5. GLOSARIO

- ARA: Application Release Automation
- ARO: Application Release Orchestration
- CDRA: Continuous Delivery and Release Automation
- CI/CD: Continuous Integration / Continuous Delivery (Integración y entrega continua)
- DevOps: Development & Operations. Conjunción de Desarrollo y operaciones TI como si se tratara de una única disciplina.
- Dev(Sec)Ops: Incorpora la dimensión de Seguridad.
- IC: Integración Continua
- SOLID: En ingeniería de software, es un acrónimo relativo al diseño SW ("Single responsibility", "Open-closed", "Liskov substitution", "Interface segregation" y "Dependency inversion")
- XL: XebiaLabs (Empresa "DevOps" con productos de integración continua)

6. TEST

6.1 PREGUNTAS DE TEST

- 1) La “entrega continua” se basa en los siguientes pilares o fundamentos:
 - a) Gestión de configuración de software.
 - b) Integración continua.
 - c) Pruebas continuas.
 - d) Todas las opciones anteriores son correctas.
- 2) ¿Qué se conoce con el término japonés jidoka?
 - a) El término correcto es ‘judoka’ y trata de aprovechar el impulso innovador ajeno en beneficio propio aplicándolo a la tecnología.
 - b) Una de las tradiciones filosóficas de Toyota, que significa aproximadamente “humanización con un toque tecnológico”.
 - c) Una de las tradiciones filosóficas de Toyota, que significa aproximadamente “automatización con un toque humano”.
 - d) Por él se entiende la realización por los humanos de tareas repetitivas para dejar que las computadoras resuelvan los auténticos problemas.
- 3) Señala la respuesta INCORRECTA. Trabajar en pequeños lotes tiene los siguientes beneficios:
 - a) Reduce el tiempo que se tarda en obtener retroalimentación sobre nuestro trabajo.
 - b) Facilita la clasificación y la solución de problemas.
 - c) Aumenta la eficiencia y la motivación.
 - d) Permite reforzar el sofisma de los costos recuperables.
- 4) Ejemplos de herramientas para la gestión de la configuración son:
 - a) CVS, Subversion o SourceSafe, ClearCase, Darcs o Bazaar.
 - b) Plastic SCM, Git, SCCS o Mercurial.
 - c) Perforce, Fossil SCM, Team Foundation Server.
 - d) Todas las herramientas de las opciones anteriores son válidas.
- 5) Señala la opción correcta:
 - a) La integración continua requiere un conjunto de pruebas unitarias automatizadas y de ejecución rápida.
 - b) Estas pruebas deben ser lo suficientemente exhaustivas como para dar un buen nivel de confianza.
 - c) Las opciones a y b son correctas.
 - d) Ninguna respuesta es del todo correcta.
- 6) Incorporar la calidad intrínsecamente en el proceso de construcción supone:
 - a) Que se deje de depender de la inspección para lograr la calidad.
 - b) Que se incorpore la necesidad de una inspección masiva al final del proceso.
 - c) Las respuestas a y b son correctas.
 - d) Realizar comprobaciones manuales de calidad dedicándoles el tiempo necesario.
- 7) En la gestión de la configuración dos objetivos primordiales son:
 - a) Responsabilidad y reproducibilidad.
 - b) Reproducibilidad y responsabilidad.
 - c) Reproducibilidad y rastreabilidad.
 - d) Cantidad y calidad van de la mano.
- 8) Señale la opción incorrecta:

- a) Una manera de detectar errores de manera rápida y eficiente es incluir pruebas automatizadas en el proceso de compilación.
 - b) Un error común es no incluir todo en la compilación automatizada.
 - c) El objetivo es contar con pruebas para encontrar la ausencia de cualquier error.
 - d) Es recomendable que los desarrolladores realicen un *commit* todos los días o incluso con mayor frecuencia.
- 9) Señale la opción correcta. Uno de los principios de la 'entrega continua' es que:
- a) Nadie del equipo es nunca del todo responsable.
 - b) No persigue sin descanso la mejora continua.
 - c) Los ordenadores resuelven los problemas.
 - d) Se trabaja en lotes pequeños.
- 10) ¿Se deben arreglar las construcciones rotas inmediatamente?
- a) Sí.
 - b) No.
 - c) Ni sí ni no, lo decide el cliente.
 - d) Kent Beck señalaba que "el equipo debe tratar de hacer una depuración inmediata en una línea principal rota", antes de seguir avanzando.
- 11) ¿Por qué la integración continua está muy relacionada con la comunicación?
- a) Porque todos pueden expresar libremente lo que piensan de sus responsables.
 - b) Porque todos pueden ver fácilmente el estado del sistema y los cambios que se le han hecho.
 - c) Porque permite que aquellos que no están ubicados en el mismo lugar puedan tener una idea del estado del proyecto.
 - d) Las opciones b y c son correctas.
- 12) En integración continua se suele citar a Kent Beck, que señalaba que:
- a) Hay que empezar por el final para saber a dónde queremos ir.
 - b) Hay que comenzar por lo que más nos duele.
 - c) Hay que avanzar con paso firme superando todas las dificultades.
 - d) Hay que encargar a los más sufridos las pruebas más duras.
- 13) Señale la respuesta INCORRECTA:
- a) La entrega continua no significa disparar a todos nuestros probadores.
 - b) No se deben detener y automatizar todas las pruebas manuales "de repente".
 - c) Se deben automatizar a la vez todas las pruebas.
 - d) Todo el equipo es responsable de las pruebas automatizadas.
- 14) Una cultura de empresa donde las nuevas ideas son bienvenidas, donde personas de toda la organización colaboran en la búsqueda de objetivos comunes o donde se traten los errores y los accidentes como oportunidades para aprender a mejorar en lugar de cazar brujas, es una organización orientada...
- a) A la promoción de los mejores.
 - b) Al rendimiento.
 - c) Al cumplimiento de las reglas.
 - d) A encontrar responsables.
- 15) Seleccione la respuesta INCORRECTA. Cuando se construye un canal de despliegue resulta valioso:
- a) Solo construir paquetes una vez: es crucial asegurarse de que lo que se está desplegando es consistente.
 - b) Desplegar de la misma manera en todos los entornos, incluido el desarrollo.
 - c) Realizar una prueba de humo (smoke test) en alguno de los despliegues.
 - d) Mantener los entornos similares, aunque pueden diferir en la configuración del hardware.

6.2 SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	D
2	C
3	D
4	D
5	C
6	A
7	C
8	C
9	D
10	A
11	D
12	B
13	C
14	B
15	C

7 BIBLIOGRAFÍA

7.1 BIBLIOGRAFÍA BÁSICA

1. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation: Jez Humble y David Farley
2. The DevOPS Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations: Gene Kim, John Willis, Patrick Debois y Jez Humble
3. Agile Testing: A Practical Guide for Testers and Agile Teams, de Lisa Crispin y Janet Gregory ISBN-13: 978-0321534460 Addison Wesley. A Michael Cohn Series Book.
4. Continuous Delivery: The Deployment Production Line Jez Humble, Chris Read, Dan North <http://www.informit.com/articles/article.aspx?p=1621865>
5. The State of DevOps Tools at <https://devops.com/>
6. <https://xebialabs.com>. La tabla periódica se ha obtenido del link: <https://xebialabs.com/periodic-table-of-devops-tools>

7.2 BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

7. *Booch, Grady (December 1998). "Object-Oriented Analysis and Design with applications (2nd edition)"*
8. *Fowler, Martin (1 May 2006). "Continuous Integration". martinfowler.com.*
9. *Beck, Kent (28 March 1998). "Extreme Programming: A Humanistic Discipline of Software Development". *Fundamental Approaches to Software Engineering: First International Conference, FASE'98, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS'98, Lisbon, Portugal, 28 March – 4 April 1998, Proceedings, Volume 1. Lisbon: Springer. p. 4. ISBN 9783540643036.**
10. Beck, Kent (1999). Extreme Programming Explained. ISBN 978-0-201-61641-5.
11. "A Brief History of DevOps, Part III: Automated Testing and Continuous Integration". *CircleCI*. 2018-02-01
12. Technical debt and agile software development practices and processes: An industry practitioner survey Information and Software Technology, Volume 96, 2018, pp. 141-160
13. Johannes Holvitie, Sherlock, A. Licorish, Rodrigo O. Spínola, Sami Hyrynsalmi, Stephen G. MacDonell, Thiago S. Mendes, Jim Buchan, Ville Leppänen
14. <https://www.thoughtworks.com/continuous-integration>
15. Cunningham, Ward (5 August 2009). "Integration Hell". *WikiWikiWeb*. <http://c2.com/cgi/wiki?IntegrationHell>
16. What is Continuous Integration? AWS <https://aws.amazon.com/devops/continuous-integration>
17. <https://lisacrispin.com/>
18. <https://agiletester.ca/>
19. The Practice of Cloud System Administration: DevOps and SRE Practices for Web Services" Thomas A. Limoncelli, Strata R. Chalup (Author), Christina J. Hogan
20. <https://paulhammant.com/2013/07/14/legacy-application-strangulation-case-studies/>
21. Release It! Michael Nygard
22. Problems, causes and solutions when adopting continuous delivery—A systematic literature review Eero Laukkanen Juha Itkonena Casper Lassenius <https://www.sciencedirect.com/science/article/pii/S0950584916302324>
23. Continuous Delivery: Overcoming adoption challenges Lianping Chen Journal of Systems and Software, Volume 128, 2017, pp. 72-86 <https://www.sciencedirect.com/science/article/pii/S0164121217300353>